

Libris - Aplicacion de Gestion de Lectura

Documentacion Completa y Manual de Usuario

Equipo de Desarrollo Libris

5 February, 2026

Table of contents

1. Libris	1
Bienvenido a Libris	2
1.1. ¿Qué es Libris?	2
1.2. ¿Para quién es esta documentación?	2
1.2.1. Usuarios Finales	2
1.2.2. Desarrolladores	3
1.2.3. Administradores	3
1.3. Tecnologías Utilizadas	3
1.4. Estructura de la Documentación	4
1.5. Comenzar	4
1.6. Contacto y Soporte	4
1.7. Versión	5
I. Manual de Usuario	6
2. Guía de Instalación	7
3. Guía de Instalación	8
3.1. Requisitos del Sistema	8
3.1.1. Para Usuarios Finales	8
3.2. Android	8
3.3. iOS	8
3.3.1. Para Desarrolladores	8
3.4. Instalación para Usuarios	10
3.4.1. Desde Google Play Store (Android)	10
3.4.2. Desde App Store (iOS)	11
3.4.3. Instalación Manual (APK - Android)	11
3.5. Configuración Inicial	11
3.5.1. Primera Ejecución	11
3.5.2. Permisos Requeridos	12
3.6. Instalación para Desarrolladores	13
3.6.1. Configurar Entorno Flutter	13
3.7. Windows	13
3.8. macOS	13

Table of contents

3.9. Linux	14
3.9.1. Configurar Android Studio	15
3.9.2. Configurar Xcode (Solo macOS)	16
3.10. Clonar y Configurar el Proyecto	16
3.10.1. 1. Clonar Repositorio	16
3.10.2. 2. Instalar Dependencias	17
3.10.3. 3. Configurar Variables de Entorno	17
3.10.4. 4. Configurar API Backend	18
3.11. Ejecutar en Desarrollo	18
3.11.1. Listar Dispositivos	18
3.11.2. Ejecutar en Emulador Android	18
3.11.3. Ejecutar en Dispositivo Físico	19
3.12. Android	19
3.13. iOS	19
3.13.1. Hot Reload y Hot Restart	19
3.14. Compilar para Producción	20
3.14.1. Android (APK/AAB)	20
3.14.2. iOS (IPA)	21
3.14.3. Web	21
3.15. Configuración del Servidor Backend	22
3.15.1. Requisitos del Servidor	22
3.15.2. Configurar Django	22
3.15.3. Base de Datos	24
3.16. Solución de Problemas	24
3.16.1. Problemas Comunes de Flutter	24
3.17. Error: SDK no encontrado	24
3.18. Error: Android License	25
3.19. Error: CocoaPods	25
3.20. Error: Gradle Build	26
3.20.1. Errores de Dependencias	26
3.20.2. Problemas de Rendimiento	26
3.21. Verificación de Instalación	27
3.21.1. Checklist Final	27
3.21.2. Script de Verificación	27
3.22. Próximos Pasos	28
4. Manual de Usuario	29
5. Manual de Usuario de Libris	30
5.1. Introducción	30
5.2. Primeros Pasos	30
5.2.1. Crear una Cuenta	30
5.3. Campos Requeridos	30

Table of contents

5.4.	Consejos	31
5.4.1.	Iniciar Sesión	31
5.5.	Navegación Principal	32
5.5.1.	Pantalla de Inicio (Home)	32
5.5.2.	Libros	32
5.5.3.	Mangas	32
5.5.4.	Novelas	32
5.5.5.	Material Educativo	32
5.5.6.	Barra de Navegación	32
5.6.	Explorar el Catálogo	32
5.6.1.	Buscar Libros	32
5.6.2.	Detalles del Libro	32
5.6.3.	Ver Portada	33
5.7.	Gestionar tus Lecturas	33
5.7.1.	Agregar a Mis Lecturas	33
5.8.	Paso 1: Seleccionar	33
5.9.	Paso 2: Agregar	34
5.10.	Paso 3: Configurar	34
5.10.1.	Ver Mis Lecturas	34
5.10.2.	Actualizar Progreso	34
5.11.	Calificaciones y Reseñas	36
5.11.1.	Calificar un Libro	36
5.11.2.	Editar tu Calificación	37
5.11.3.	Eliminar tu Calificación	37
5.12.	Comentarios	37
5.12.1.	Dejar un Comentario	37
5.13.	Escribir	37
5.14.	Publicar	37
5.15.	Consejos	38
5.15.1.	Ver Comentarios	38
5.15.2.	Editar tu Comentario	38
5.15.3.	Eliminar tu Comentario	38
5.16.	Lector de PDF	40
5.16.1.	Abrir un PDF	40
5.16.2.	Navegar en el PDF	41
5.16.3.	Controles Básicos	41
5.16.4.	Funciones Avanzadas	41
5.17.	Tu Perfil	41
5.17.1.	Ver Información de Perfil	41
5.17.2.	Editar Perfil	42
5.17.3.	Cerrar Sesión	42
5.18.	Consejos y Trucos	42
5.18.1.	Maximiza tu Experiencia	42
5.19.	Organización	42

Table of contents

5.20. Exploración	42
5.21. Lectura	42
5.22. Comunidad	43
5.23. Solución de Problemas Comunes	43
5.23.1. No puedo iniciar sesión	43
5.23.2. Los libros no cargan	45
5.23.3. El PDF no se abre	46
5.23.4. La app se cierra inesperadamente	46
5.24. Preguntas Frecuentes (FAQ)	46
5.24.1. General	46
5.24.2. Lecturas	47
5.24.3. Calificaciones y Comentarios	47
5.24.4. Cuenta	47
5.25. Conclusión	47
6. Preguntas Frecuentes	49
7. Preguntas Frecuentes (FAQ)	50
7.1. Sobre la Aplicación	50
7.1.1. ¿Qué es Libris?	50
7.1.2. ¿Libris es gratuito?	50
7.1.3. ¿En qué plataformas está disponible Libris?	50
7.1.4. ¿Necesito crear una cuenta para usar Libris?	50
7.2. Cuenta y Seguridad	51
7.2.1. ¿Cómo creo una cuenta?	51
7.2.2. ¿Olvidé mi contraseña, qué hago?	51
7.2.3. ¿Mis datos están seguros?	51
7.2.4. ¿Puedo usar la misma cuenta en múltiples dispositivos?	51
7.2.5. ¿Cómo elimino mi cuenta?	52
7.3. Catálogo y Contenido	52
7.3.1. ¿Cuántos libros hay en el catálogo?	52
7.3.2. ¿Puedo agregar mis propios libros?	52
7.3.3. ¿Los libros son completos o solo muestras?	53
7.3.4. ¿Puedo descargar libros para leer sin conexión?	53
7.3.5. ¿Cómo sé si un libro tiene PDF disponible?	53
7.4. Lectura y Progreso	53
7.4.1. ¿Cómo agrego un libro a mis lecturas?	53
7.4.2. ¿Puedo cambiar el estado de una lectura?	54
7.4.3. ¿Se guarda automáticamente mi progreso?	54
7.4.4. ¿Libris recuerda en qué página me quedé?	54
7.4.5. ¿Puedo eliminar un libro de mis lecturas?	54
7.5. Calificaciones y Comentarios	55
7.5.1. ¿Cómo califico un libro?	55
7.5.2. ¿Puedo cambiar mi calificación?	55

Table of contents

7.5.3.	¿Puedo calificar sin haber leído el libro?	55
7.5.4.	¿Cómo elimino mi calificación?	55
7.5.5.	¿Cómo dejo un comentario?	55
7.5.6.	¿Puedo editar mi comentario después de publicarlo?	56
7.5.7.	¿Los comentarios tienen límite de caracteres?	56
7.5.8.	¿Puedo ver quién calificó o comentó?	56
7.5.9.	¿Puedo reportar comentarios inapropiados?	56
7.6.	Lector de PDF	56
7.6.1.	¿Qué formatos de archivo soporta Libris?	56
7.6.2.	¿Cómo abro el lector de PDF?	57
7.6.3.	¿Puedo hacer zoom en el PDF?	57
7.6.4.	¿Cómo navego entre páginas?	57
7.6.5.	¿El lector tiene modo nocturno?	57
7.6.6.	¿Puedo rotar la pantalla al leer?	57
7.6.7.	¿Puedo compartir el PDF?	58
7.6.8.	El PDF no carga, ¿qué hago?	58
7.7.	Conexión y Sincronización	58
7.7.1.	¿Necesito Internet para usar Libris?	58
7.7.2.	¿Funciona con WiFi y datos móviles?	58
7.7.3.	¿Qué pasa si pierdo la conexión mientras leo?	59
7.7.4.	¿Mis datos se sincronizan entre dispositivos?	59
7.7.5.	¿Con qué frecuencia se sincroniza?	59
7.8.	Configuración y Personalización	59
7.8.1.	¿Puedo cambiar el tema (claro/oscuro)?	59
7.8.2.	¿Puedo cambiar el tamaño de fuente?	59
7.8.3.	¿Puedo cambiar mi nombre de usuario?	60
7.8.4.	¿Puedo cambiar mi email?	60
7.8.5.	¿Dónde están las notificaciones?	60
7.9.	Problemas Técnicos	60
7.9.1.	La app se cierra inesperadamente	60
7.9.2.	No puedo iniciar sesión	61
7.9.3.	Los libros no cargan o aparecen en blanco	62
7.9.4.	El lector PDF está en blanco o no carga	62
7.9.5.	Mis calificaciones o comentarios no se guardan	62
7.9.6.	Errores de “Token expirado” o “No autorizado”	63
7.10.	Rendimiento y Datos	63
7.10.1.	¿Cuánto espacio ocupa Libris?	63
7.10.2.	¿Cuántos datos consume Libris?	63
7.10.3.	¿Libris almacena datos en caché?	64
7.10.4.	¿Cómo limpio la caché?	64
7.11.	Actualizaciones	64
7.11.1.	¿Cómo sé si hay una actualización disponible?	64
7.11.2.	¿Las actualizaciones son automáticas?	65
7.11.3.	¿Perderé mis datos al actualizar?	65

Table of contents

7.11.4. ¿Con qué frecuencia se actualiza Libris?	65
7.12. Soporte y Ayuda	65
7.12.1. ¿Cómo contacto al soporte?	65
7.12.2. ¿Qué información debo proporcionar al reportar un problema?	65
7.12.3. ¿Puedo solicitar nuevas funciones?	66
7.12.4. ¿Cómo puedo contribuir al proyecto?	66
7.13. Futuras Funciones	67
7.13.1. ¿Qué funciones están planificadas?	67
7.13.2. ¿Cuándo estará disponible el modo offline?	67
7.13.3. ¿Habrá soporte para otros formatos (EPUB, MOBI)?	67
7.13.4. ¿Libris estará disponible en web?	67
7.14. Más Información	67
II. Documentacion Tecnica	69
8. Documentación Técnica	70
9. Documentación Técnica	71
9.1. Descripción General del Proyecto	71
9.1.1. Stack Tecnológico	71
9.2. Arquitectura de la Aplicación	72
9.2.1. Patrón de Arquitectura	72
9.2.2. Capas de la Aplicación	73
9.3. Estructura del Proyecto	73
9.4. Flujo de Autenticación	76
9.4.1. Sistema JWT	76
9.4.2. Implementación	77
9.5. Gestión de Estado	78
9.5.1. Provider Pattern	78
9.5.2. Ejemplo de Provider	78
9.6. Modelos de Datos	80
9.6.1. Diagrama de Clases	80
9.6.2. Serialización JSON	80
9.7. Cliente API (Dio)	81
9.7.1. Configuración	81
9.7.2. Manejo de Errores	82
9.8. Almacenamiento Local	83
9.8.1. Flutter Secure Storage	83
9.9. Tematización	84
9.9.1. Tema de la Aplicación	84
9.10. Navegación	85
9.10.1. Sistema de Rutas	85

Table of contents

9.11. Testing	85
9.11.1. Estructura de Tests	85
9.11.2. Ejemplo de Test	86
9.12. Performance y Optimización	86
9.12.1. Buenas Prácticas Implementadas	86
9.12.2. Ejemplo de Optimización	87
9.13. Seguridad	88
9.13.1. Medidas de Seguridad Implementadas	88
9.13.2. Validación de Inputs	88
9.14. Configuración Multi-entorno	89
9.14.1. Entornos	89
9.15. Build y CI/CD	89
9.15.1. Scripts de Build	89
9.16. Conclusión	90
10.API Reference	91
11.API Reference	92
11.1. Base URL	92
11.2. Autenticación	92
11.2.1. Sistema JWT	92
11.2.2. Headers Requeridos	92
11.3. Endpoints	93
11.3.1. 1. Autenticación	93
11.3.2. 2. Catálogo	97
11.3.3. 3. Mis Lecturas (Registros)	99
11.3.4. 4. Calificaciones	102
11.3.5. 5. Comentarios	104
11.4. Códigos de Estado HTTP	107
11.4.1. Exitosos (2xx)	107
11.4.2. Errores del Cliente (4xx)	107
11.4.3. Errores del Servidor (5xx)	107
11.5. Manejo de Errores	108
11.5.1. Formato de Error	108
11.5.2. Ejemplo en Flutter	108
11.6. Rate Limiting	109
11.7. Paginación	109
11.8. Búsqueda y Filtrado	110
11.9. CORS	110
11.10Versionado	111
11.11Recursos Adicionales	111
11.12Changelog	111
11.12.1.v1.0.0 (2024-02-05)	111

12. Modelos de Datos	112
13. Modelos de Datos	113
13.1. Introducción	113
13.2. Modelos Principales	113
13.2.1. User (Usuario)	113
13.2.2. Book (Libro/Manga/Novela/Material)	115
13.2.3. ReadingRecord (Registro de Lectura)	117
13.2.4. Rating (Calificación)	120
13.2.5. Comment (Comentario)	123
13.3. Diagramas de Relaciones	127
13.3.1. Diagrama ER	127
13.4. Validaciones	128
13.4.1. User	128
13.4.2. Rating	129
13.4.3. Comment	129
13.5. Serialización y Deserialización	130
13.5.1. Manejo de Nullables	130
13.5.2. JSON Encoding	130
13.6. Constantes y Enums	130
13.7. Recursos Adicionales	131
14. Arquitectura	132
15. Arquitectura del Sistema	133
15.1. Visión General	133
15.2. Patrón MVVM	135
15.3. Gestión de Estado con Provider	135
15.4. Flujo de Datos	137
15.4.1. Lectura de Datos (Query)	137
15.4.2. Escritura de Datos (Mutation)	138
15.5. Inyección de Dependencias	138
15.6. Navegación	139
15.7. Seguridad	139
15.7.1. Almacenamiento Seguro de Tokens	139
15.8. Manejo de Errores	140
15.9. Caché y Optimización	141
15.10. Testing	141
15.10.1. Estructura de Tests	141
15.10.2. Mocking	142

III. Guías de Desarrollo	143
16. Guía de Desarrollo	144
17. Guía de Desarrollo	145
17.1. Prerrequisitos	145
17.2. Clonar el Repositorio	145
17.3. Estructura de Branches	145
17.4. Workflow de Desarrollo	146
17.5. Convenciones de Código	146
17.5.1. Formato	146
17.5.2. Nombrado	146
17.5.3. Imports	147
17.6. Testing	147
17.6.1. Ejecutar Tests	147
17.6.2. Escribir Tests	148
17.7. Debug	148
17.7.1. Ejecutar en modo debug	148
17.7.2. Hot Reload	148
17.7.3. DevTools	148
17.8. Build	149
17.8.1. Debug Build	149
17.8.2. Release Build	149
17.9. Contribuir	149
18. Solución de Problemas	150
19. Troubleshooting	151
19.1. Problemas de Instalación	151
19.1.1. Flutter Doctor muestra errores	151
19.2. Problemas de Compilación	151
19.2.1. Gradle build failed	151
19.2.2. Pod install failed (iOS)	151
19.3. Problemas de Red	152
19.3.1. Connection timeout	152
19.3.2. 401 Unauthorized constante	152
19.4. Problemas de Estado	152
19.4.1. UI no se actualiza	152
19.5. Logs y Debugging	152
19.6. Obtener Ayuda	153
20. Guía de Contribución	154
21. Contributing Guide	155
21.1. Código de Conducta	155

Table of contents

21.2. Cómo Contribuir	155
21.2.1. Reportar Bugs	155
21.2.2. Solicitar Features	155
21.2.3. Pull Requests	156
21.3. Convenciones de Commit	156
21.4. Checklist para PR	157
21.5. Revisión de Código	157
21.6. Contacto	157

1. Libris

Tu compañero de lectura digital

Bienvenido a Libris

1.1. ¿Qué es Libris?

Libris es una aplicación móvil moderna desarrollada en Flutter que te permite gestionar tu biblioteca personal de lectura. Con Libris puedes:

Características Principales

- **Catálogo Extenso:** Accede a libros, mangas, novelas y material educativo
- **Seguimiento de Lectura:** Registra tu progreso de lectura en tiempo real
- **Calificaciones:** Valora tus lecturas del 1 al 5 estrellas
- **Comentarios:** Comparte tus opiniones sobre cada libro
- **Lector PDF:** Lee tus documentos directamente en la app
- **Seguro:** Autenticación JWT y almacenamiento seguro

1.2. ¿Para quién es esta documentación?

Esta documentación está diseñada para diferentes tipos de usuarios:

1.2.1. Usuarios Finales

Si eres un usuario que quiere usar la aplicación móvil, consulta el [Manual de Usuario](#) donde encontrarás:

- Cómo instalar la app
- Cómo crear una cuenta
- Cómo buscar y agregar libros
- Cómo usar el lector PDF
- Preguntas frecuentes

1.2.2. Desarrolladores

Si eres desarrollador y quieres entender cómo funciona la aplicación o contribuir al proyecto, consulta:

- [Documentación Técnica](#): Arquitectura y estructura del código
- [API Reference](#): Endpoints REST y ejemplos de uso
- [Guía de Desarrollo](#): Cómo configurar el entorno de desarrollo

1.2.3. Administradores

Si necesitas configurar o mantener el sistema:

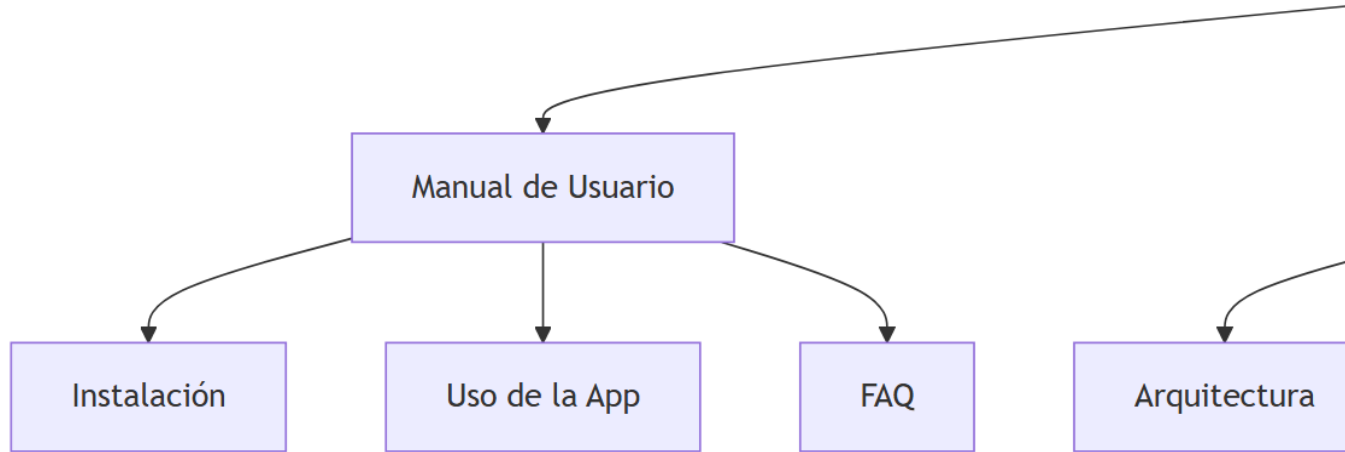
- [Guía de Instalación](#): Requisitos y configuración
- [Solución de Problemas](#): Errores comunes y soluciones

1.3. Tecnologías Utilizadas

Libris está construido con tecnologías modernas y confiables:

Tecnología	Uso
Flutter 3.10.7	Framework multiplataforma
Dart	Lenguaje de programación
Provider	Gestión de estado
Dio	Cliente HTTP
JWT	Autenticación
REST API	Backend
PDFx	Visor de PDF

1.4. Estructura de la Documentación



1.5. Comenzar

! Por Donde Empezar

- ¿Primera vez usando Libris? → Ve al [Manual de Usuario](#)
- ¿Quieres desarrollar? → Empieza con [Instalación](#)
- ¿Necesitas la API? → Consulta [API Reference](#)

1.6. Contacto y Soporte

¿Tienes preguntas o necesitas ayuda?

- Email: soporte@libris.app
- Slack: [Únete a nuestro workspace](#)
- Issues: [GitHub Issues](#)
- Docs: Esta documentación

1.7. Versión

i Note

Versión de la App: 1.0.0+1

Última actualización de docs: `r Sys.Date()`

Flutter SDK: 3.10.7

Hecho con por el equipo de Libris

Part I.

Manual de Usuario

2. Guía de Instalación

Configuración del entorno de desarrollo y despliegue

3. Guía de Instalación

3.1. Requisitos del Sistema

3.1.1. Para Usuarios Finales

3.2. Android

Requisitos mínimos: - Android 5.0 (Lollipop) o superior - 100 MB de espacio libre - Conexión a Internet

Recomendado: - Android 8.0 (Oreo) o superior - 200 MB de espacio libre - WiFi para descargas

3.3. iOS

Requisitos mínimos: - iOS 11.0 o superior - iPhone 6s o posterior - 100 MB de espacio libre - Conexión a Internet

Recomendado: - iOS 14.0 o superior - 200 MB de espacio libre - WiFi para descargas

3.3.1. Para Desarrolladores

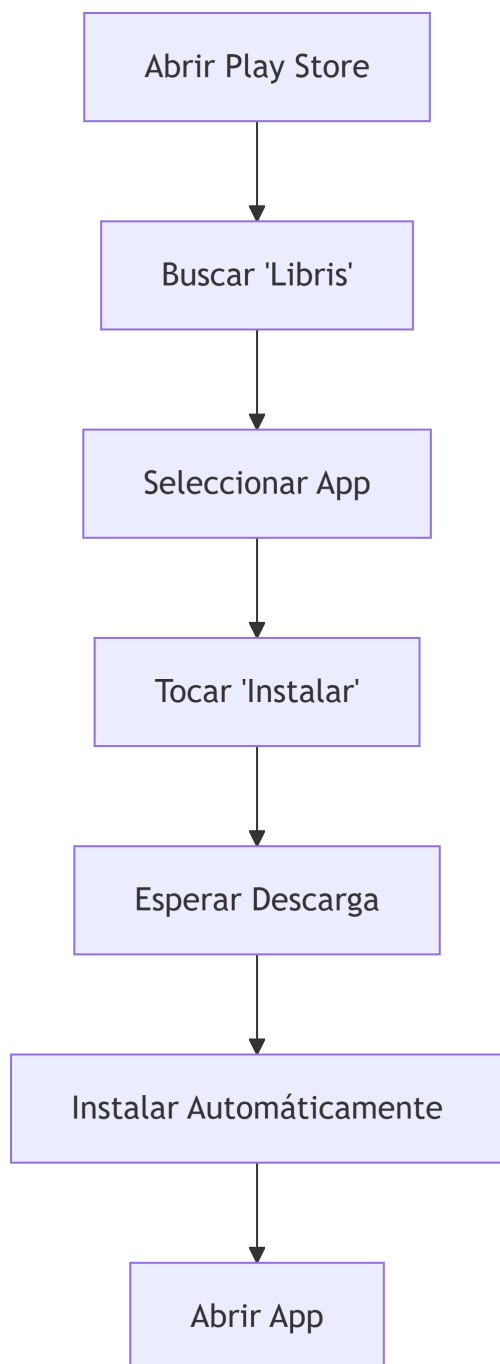
! Requisitos de Desarrollo

Componente	Versión Requerida
Flutter SDK	3.10.7 o superior
Dart SDK	Incluido con Flutter
Android Studio	2021.3.1 o superior
Xcode	14.0+ (solo macOS)
Git	2.0+
RAM	Mínimo 8 GB
Espacio en Disco	10 GB libres

3. *Guía de Instalación*

3.4. Instalación para Usuarios

3.4.1. Desde Google Play Store (Android)



3. Guía de Instalación

1. Abre **Google Play Store**
2. Busca “**Libris**”
3. Toca “**Instalar**”
4. Espera a que termine la instalación
5. Toca “**Abrir**”

3.4.2. Desde App Store (iOS)

1. Abre **App Store**
2. Busca “**Libris**”
3. Toca “**Obtener**”
4. Auténticate con Face ID/Touch ID/Contraseña
5. Espera la instalación
6. Toca “**Abrir**”

3.4.3. Instalación Manual (APK - Android)

 Solo para Pruebas

Este método es solo para distribución interna o pruebas. Para producción, usa las tiendas oficiales.

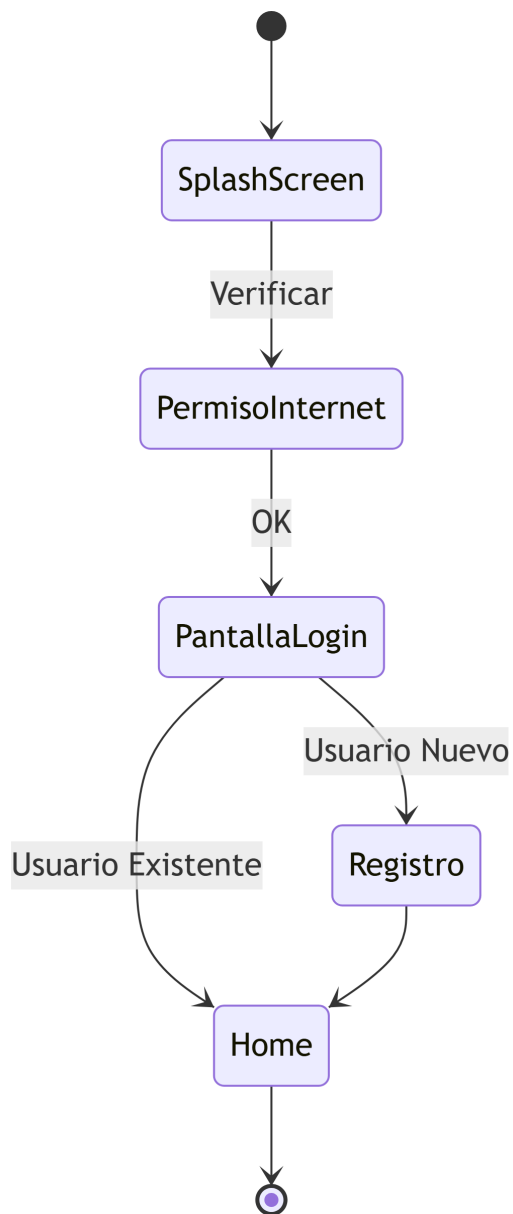
Pasos:

1. **Descarga el APK** desde el servidor o repositorio autorizado
 2. **Habilita instalación de fuentes desconocidas:**
 - Ve a Configuración → Seguridad
 - Activa “Fuentes desconocidas” o “Instalar apps desconocidas”
 3. **Abre el archivo APK** descargado
 4. Toca “**Instalar**”
 5. **Abre la app** una vez instalada
-

3.5. Configuración Inicial

3.5.1. Primera Ejecución

Cuando abras Libris por primera vez:



1. **Pantalla de Bienvenida** se mostrará brevemente
2. **Verificación de conexión** a Internet
3. **Pantalla de Login** para iniciar sesión o registrarse

3.5.2. Permisos Requeridos

La app solicitará los siguientes permisos:

3. Guía de Instalación

Permiso	Uso	¿Obligatorio?
Internet	Acceder a la API	Sí
Almacenamiento	Guardar preferencias locales	Sí
Red	Verificar conectividad	Sí

Privacidad

Libris no solicita permisos de cámara, micrófono, ubicación ni contactos. Solo usa lo mínimo necesario para funcionar.

3.6. Instalación para Desarrolladores

3.6.1. Configurar Entorno Flutter

3.6.1.1. 1. Instalar Flutter SDK

3.7. Windows

```
# Descargar Flutter SDK
Invoke-WebRequest -Uri "https://storage.googleapis.com/flutter_infra_release/releases/stable/windows/flutter_windows_v1.10.0-post.0.0.zip" -OutFile flutter.zip

# Extraer
Expand-Archive flutter.zip -DestinationPath C:\src

# Agregar al PATH
$env:Path += ";C:\src\flutter\bin"
[System.Environment]::SetEnvironmentVariable("Path", $env:Path, [System.EnvironmentVariableTarget]::Machine)

# Verificar instalación
flutter doctor
```

3.8. macOS

3. Guía de Instalación

```
# Instalar con Homebrew
brew install --cask flutter

# O descargar manualmente
cd ~/development
unzip ~/Downloads/flutter_macos_3.10.7-stable.zip

# Agregar al PATH en ~/.zshrc o ~/.bash_profile
export PATH="$PATH:$HOME/development/flutter/bin"

# Aplicar cambios
source ~/.zshrc

# Verificar instalación
flutter doctor
```

3.9. Linux

```
# Descargar Flutter
cd ~/development
wget https://storage.googleapis.com/flutter_infra_release/releases/stable/linux/flutter_

# Extraer
tar xf flutter_linux_3.10.7-stable.tar.xz

# Agregar al PATH en ~/.bashrc
export PATH="$PATH:$HOME/development/flutter/bin"

# Aplicar cambios
source ~/.bashrc

# Verificar instalación
flutter doctor
```

3.9.0.1. 2. Ejecutar Flutter Doctor

```
flutter doctor
```

Salida esperada:

3. Guía de Instalación

Doctor summary (to see all details, run `flutter doctor -v`):

```
[ ] Flutter (Channel stable, 3.10.7, on Windows 11, locale es-ES)
[ ] Android toolchain - develop for Android devices
[ ] Chrome - develop for the web
[ ] Visual Studio Code (version 1.85.0)
[ ] Connected device (2 available)
[ ] Network resources
```

💡 Resolver Problemas

Si algún ítem muestra [] o [!], ejecuta `flutter doctor -v` para ver detalles y sigue las instrucciones.

3.9.1. Configurar Android Studio

3.9.1.1. 1. Descargar e Instalar

1. Descarga desde developer.android.com/studio
2. Ejecuta el instalador
3. Sigue el asistente de instalación

3.9.1.2. 2. Instalar Android SDK



En Android Studio:

1. Ve a **Tools** → **SDK Manager**
2. En **SDK Platforms**, marca:
 - Android 13.0 (API 33)
 - Android 12.0 (API 31)
 - Android 11.0 (API 30)
 - Android 10.0 (API 29)
3. En **SDK Tools**, verifica:
 - Android SDK Build-Tools
 - Android SDK Command-line Tools
 - Android Emulator
 - Android SDK Platform-Tools

3. Guía de Instalación

4. Toca **“Apply”** y espera la descarga

3.9.1.3. 3. Instalar Flutter Plugin

1. Ve a **File → Settings → Plugins**
2. Busca **“Flutter”**
3. Toca **“Install”**
4. Reinicia Android Studio

3.9.2. Configurar Xcode (Solo macOS)

```
# Instalar Xcode desde App Store
# Luego ejecutar:

sudo xcode-select --switch /Applications/Xcode.app/Contents/Developer
sudo xcodebuild -runFirstLaunch

# Aceptar licencia
sudo xcodebuild -license accept

# Instalar CocoaPods
sudo gem install cocoapods
```

3.10. Clonar y Configurar el Proyecto

3.10.1. 1. Clonar Repositorio

```
# Clonar desde GitHub
git clone https://github.com/tu-usuario/libris.git

# Navegar al directorio
cd libros/libris
```

3.10.2. 2. Instalar Dependencias

```
# Obtener paquetes de Flutter
flutter pub get
```

Dependencias principales:

```
dependencies:
  flutter:
    sdk: flutter
  provider: ^6.1.5+1      # Gestión de estado
  dio: ^5.4.3+1          # Cliente HTTP
  flutter_secure_storage: ^9.0.0 # Almacenamiento seguro
  url_launcher: ^6.1.0    # Abrir URLs
  file_picker: ^8.0.5     # Selector de archivos
  google_fonts: ^6.2.0    # Fuentes de Google
  pdfx: ^2.9.2            # Visor PDF
  path_provider: ^2.1.1   # Rutas del sistema
  http: ^1.2.0            # HTTP adicional
```

3.10.3. 3. Configurar Variables de Entorno

Crea un archivo `.env` en la raíz del proyecto:

```
# API Configuration
API_BASE_URL=https://tu-servidor.com/api
API_TIMEOUT=30000

# JWT Configuration
JWT_SECRET=tu_secreto_jwt_aqui
TOKEN_EXPIRY=3600

# App Configuration
APP_NAME=Libris
APP_VERSION=1.0.0
```

Seguridad

Nunca commites el archivo `.env` al repositorio. Agrégalo a `.gitignore`.

3.10.4. 4. Configurar API Backend

Edita `lib/api/api_client.dart` con la URL correcta:

```
class ApiClient {  
  static const String baseUrl = 'https://tu-servidor.com/api';  
  
  // ... resto del código  
}
```

3.11. Ejecutar en Desarrollo

3.11.1. Listar Dispositivos

```
# Ver dispositivos conectados  
flutter devices
```

Salida ejemplo:

2 connected devices:

sdk gphone64 arm64 (mobile)	• emulator-5554	• android-arm64	• Android 13 (API 33)
Chrome (web)	• chrome	• web-javascript	• Google Chrome 119

3.11.2. Ejecutar en Emulador Android

```
# Iniciar emulador (si no está corriendo)  
flutter emulators --launch <emulator_id>  
  
# Ejecutar app en modo debug  
flutter run  
  
# Ejecutar en dispositivo específico  
flutter run -d emulator-5554
```

3.11.3. Ejecutar en Dispositivo Físico

3.12. Android

1. Habilita Opciones de Desarrollador:

- Ve a Configuración → Acerca del teléfono
- Toca “Número de compilación” 7 veces

2. Habilita Depuración USB:

- Ve a Configuración → Opciones de desarrollador
- Activa “Depuración USB”

3. Conecta el dispositivo vía USB

4. Acepta el mensaje de depuración USB en el teléfono

5. Ejecuta:

```
flutter run
```

3.13. iOS

1. Conecta el iPhone vía USB al Mac

2. Confía en la computadora en el iPhone

3. Abre el proyecto en Xcode:

```
open ios/Runner.xcworkspace
```

4. Selecciona tu dispositivo en Xcode

5. Configura firma (Team en Signing & Capabilities)

6. Ejecuta:

```
flutter run
```

3.13.1. Hot Reload y Hot Restart

Durante el desarrollo:

3. Guía de Instalación

```
# Hot Reload (recarga cambios sin perder estado)
r

# Hot Restart (reinicia la app)
R

# Abrir DevTools
v

# Salir
q
```

3.14. Compilar para Producción

3.14.1. Android (APK/AAB)

3.14.1.1. APK (Para distribución directa)

```
# Release APK
flutter build apk --release

# Split APKs por ABI (tamaño menor)
flutter build apk --split-per-abi
```

Salida: build/app/outputs/flutter-apk/app-release.apk

3.14.1.2. AAB (Para Google Play Store)

```
# Release AAB
flutter build appbundle --release
```

Salida: build/app/outputs/bundle/release/app-release.aab

💡 Firmar la App

Para producción, configura el signing en `android/app/build.gradle.kts`:

```
signingConfigs {  
    create("release") {  
        storeFile = file("../key.jks")  
        storePassword = "password"  
        keyAlias = "key"  
        keyPassword = "password"  
    }  
}  
  
buildTypes {  
    release {  
        signingConfig = signingConfigs.getByName("release")  
    }  
}
```

3.14.2. iOS (IPA)

```
# Build para iOS  
flutter build ios --release  
  
# Abrir Xcode para archivar  
open ios/Runner.xcworkspace
```

En Xcode:

1. Product → Archive
2. Espera la compilación
3. Window → Organizer
4. Selecciona el archivo
5. Distribute App → App Store Connect
6. Sigue el asistente

3.14.3. Web


```
# Build para Web
flutter build web --release

# Servidor de prueba local
cd build/web
python -m http.server 8000
```

Abre: <http://localhost:8000>

3.15. Configuración del Servidor Backend

3.15.1. Requisitos del Servidor

```
Backend: Django REST Framework
Python: 3.8+
Database: PostgreSQL 12+
Cache: Redis (opcional)
```

3.15.2. Configurar Django

3.15.2.1. 1. Instalar Dependencias

```
pip install django djangorestframework
pip install djangorestframework-simplejwt
pip install django-cors-headers
pip install psycopg2-binary
```

3.15.2.2. 2. Configurar CORS

En `settings.py`:

3. Guía de Instalación

```
INSTALLED_APPS = [  
    # ...  
    'rest_framework',  
    'corsheaders',  
]  
  
MIDDLEWARE = [  
    'corsheaders.middleware.CorsMiddleware',  
    # ...  
]  
  
# Permitir Flutter app  
CORS_ALLOWED_ORIGINS = [  
    "http://localhost:8080",  
    "https://tu-dominio.com",  
]  
  
# 0 para desarrollo  
CORS_ALLOW_ALL_ORIGINS = True # Solo desarrollo
```

3.15.2.3. 3. Configurar JWT

```
REST_FRAMEWORK = {  
    'DEFAULT_AUTHENTICATION_CLASSES': [  
        'rest_framework_simplejwt.authentication.JWTAuthentication',  
    ],  
}  
  
SIMPLE_JWT = {  
    'ACCESS_TOKEN_LIFETIME': timedelta(hours=1),  
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),  
}
```

3.15.2.4. 4. URLs del API

```
from django.urls import path, include  
  
urlpatterns = [  
    path('api/', include('tu_app.urls')),  
]
```

3.15.3. Base de Datos

```
# Crear base de datos PostgreSQL
createdb libris_db

# Configurar en settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'libris_db',
        'USER': 'postgres',
        'PASSWORD': 'tu_password',
        'HOST': 'localhost',
        'PORT': '5432',
    }
}

# Migraciones
python manage.py makemigrations
python manage.py migrate

# Crear superusuario
python manage.py createsuperuser
```

3.16. Solución de Problemas

3.16.1. Problemas Comunes de Flutter

3.17. Error: SDK no encontrado

Síntoma:

Error: Unable to find git in your PATH.

Solución:

```
# Windows
choco install git

# macOS
brew install git

# Linux
sudo apt-get install git
```

3.18. Error: Android License

Síntoma:

```
[!] Android toolchain - develop for Android devices
    Android license status unknown.
```

Solución:

```
flutter doctor --android-licenses
# Acepta todas las licencias (y)
```

3.19. Error: CocoaPods

Síntoma:

Error running pod install

Solución:

```
cd ios
pod repo update
pod install
cd ..
flutter clean
flutter pub get
```

3.20. Error: Gradle Build

Síntoma:

FAILURE: Build failed with an exception.

Solución:

```
cd android
./gradlew clean
cd ..
flutter clean
flutter pub get
flutter run
```

3.20.1. Errores de Dependencias

```
# Limpiar caché de Flutter
flutter clean

# Actualizar dependencias
flutter pub upgrade

# Reparar pub cache
flutter pub cache repair

# Reinstalar todo
rm -rf pubspec.lock
flutter pub get
```

3.20.2. Problemas de Rendimiento

Optimización

1. Usa perfiles de release:

```
flutter run --profile
```

2. Analiza performance:

```
flutter run --trace-startup --profile
```

3. Reduce tamaño del APK:

```
flutter build apk --split-per-abi --release
```

3.21. Verificación de Instalación

3.21.1. Checklist Final

Antes de considerar la instalación completa, verifica:

- ☐ Flutter doctor muestra todas las marcas verdes
- ☐ Puedes ejecutar `flutter run` exitosamente
- ☐ La app se conecta al backend correctamente
- ☐ Puedes iniciar sesión en la app
- ☐ Los libros cargan correctamente
- ☐ El lector PDF funciona
- ☐ Las calificaciones y comentarios se guardan
- ☐ No hay errores en la consola

3.21.2. Script de Verificación

```
#!/bin/bash

echo " Verificando instalación de Libris..."

# Check Flutter
if command -v flutter &> /dev/null; then
    echo " Flutter instalado"
    flutter --version
else
    echo " Flutter no encontrado"
    exit 1
fi

# Check dependencies
```

3. Guía de Instalación

```
echo " Verificando dependencias..."
cd libris/libris
flutter pub get

# Run tests
echo " Ejecutando tests..."
flutter test

# Check API connection
echo " Verificando conexión al API..."
curl -f https://tu-servidor.com/api/ || echo " API no disponible"

echo " Verificación completa"
```

3.22. Próximos Pasos

Una vez instalado exitosamente:

1. Lee el [Manual de Usuario](#)
2. Consulta la [Documentación Técnica](#)
3. Revisa la [API Reference](#)
4. Comienza a desarrollar

Soporte

Si encuentras problemas durante la instalación, consulta [Troubleshooting](#) o contacta al equipo de desarrollo.

4. Manual de Usuario

Guía completa para usar la aplicación móvil Libris

5. Manual de Usuario de Libris

5.1. Introducción

Bienvenido al manual de usuario de Libris. Esta guía te ayudará a sacar el máximo provecho de tu aplicación de gestión de lectura.

5.2. Primeros Pasos

5.2.1. Crear una Cuenta

1. **Abrir la aplicación** Libris en tu dispositivo móvil
2. **Toca “Registrarse”** en la pantalla de inicio
3. **Completa el formulario** con tus datos:

5.3. Campos Requeridos

Campo	Descripción	Ejemplo
Usuario	Tu nombre de usuario único	juan_lector
Email	Correo electrónico válido	juan@email.com
Contraseña	Mínimo 8 caracteres	• • • • • • • •
Nombre	Tu nombre real	Juan
Apellido	Tu apellido	Pérez

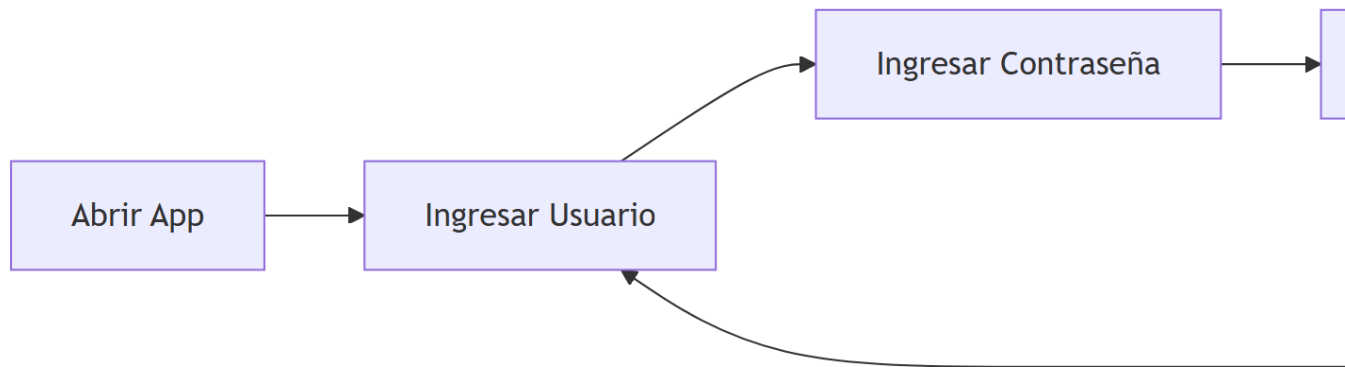
5.4. Consejos

💡 Crea una contraseña segura

- Usa al menos 8 caracteres
- Combina letras mayúsculas y minúsculas
- Incluye números y símbolos
- No uses información personal

4. Toca **“Registrar”** para crear tu cuenta
5. ¡**Listo!** Ya puedes iniciar sesión

5.4.1. Iniciar Sesión



1. Abre la aplicación
2. Ingresa tu **usuario** y **contraseña**
3. Toca **“Iniciar Sesión”**

⚠️ ¿Olvidaste tu contraseña?

Contacta al administrador del sistema para recuperar tu cuenta.

5.5. Navegación Principal

5.5.1. Pantalla de Inicio (Home)

La pantalla principal muestra cuatro secciones:

5.5.2. Libros

Explora el catálogo de libros disponibles

5.5.4. Novelas

Navega por las novelas disponibles

5.5.3. Mangas

Descubre la colección de mangas

5.5.5. Material Educativo

Accede a material de estudio y educativo

5.5.6. Barra de Navegación

Icono	Función	Descripción
	Inicio	Vuelve a la pantalla principal
	Perfil	Ver y editar tu perfil de usuario
	Configuración	Ajustes de la aplicación

5.6. Explorar el Catálogo

5.6.1. Buscar Libros

1. **Selecciona una categoría** (Libros, Mangas, Novelas o Material)
2. **Navega por la lista** desplazando hacia abajo
3. **Toca un libro** para ver sus detalles

5.6.2. Detalles del Libro

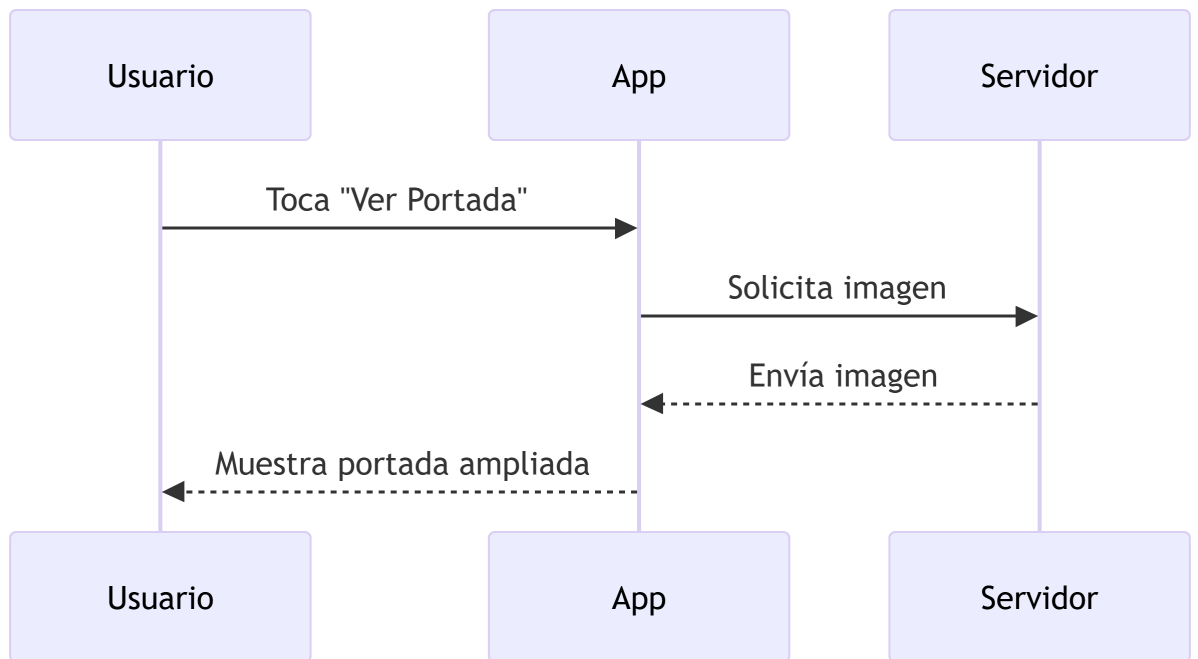
Cuando seleccionas un libro, verás:

Información Disponible

- **Portada** del libro
- **Título y Autor**
- **Descripción** completa
- **Calificación promedio**

- **Comentarios** de otros usuarios
- **Botón para leer** (si es PDF)

5.6.3. Ver Portada



Toca **"Ver Portada"** para ver la imagen en pantalla completa.

5.7. Gestionar tus Lecturas

5.7.1. Agregar a Mis Lecturas

5.8. Paso 1: Seleccionar

Toca el libro que quieres agregar

5.9. Paso 2: Agregar

Presiona el botón “**Agregar a Mis Lecturas**”

5.10. Paso 3: Configurar

Define el estado inicial: - **Leyendo** - En progreso - **Pendiente** - Para leer después - **Completado** - Ya leído

5.10.1. Ver Mis Lecturas

1. Ve a tu **Perfil**
2. Sección “**Mis Lecturas**”
3. Verás todos tus libros registrados

5.10.2. Actualizar Progreso

Para actualizar el progreso de un libro:

1. **Accede a “Mis Lecturas”**
2. **Toca el libro** que estás leyendo
3. **Actualiza el estado:**
 - Cambia entre: Leyendo → Completado
 - O marca como Pendiente

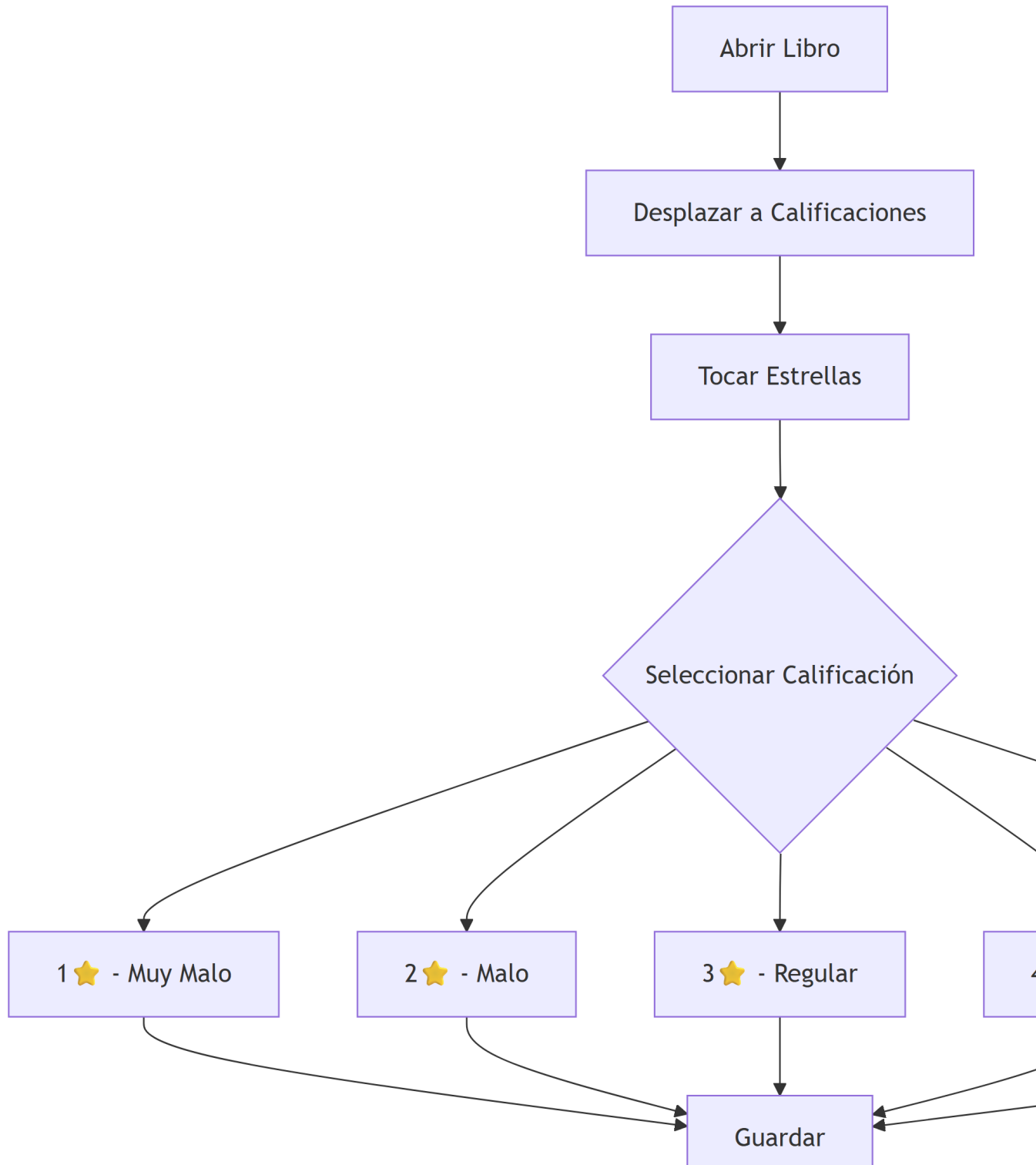
 Mantén tu lista actualizada

Actualiza regularmente el estado de tus lecturas para tener un seguimiento preciso de tu progreso.

5. *Manual de Usuario de Libris*

5.11. Calificaciones y Reseñas

5.11.1. Calificar un Libro



Pasos:

1. Abre la página de detalles del libro
2. Desplázate hasta la sección **“Calificaciones”**
3. **Toca las estrellas** para calificar (1-5)
4. Tu calificación se guarda automáticamente

5.11.2. Editar tu Calificación

- **Toca nuevamente las estrellas** para cambiar tu calificación
- La nueva calificación reemplaza la anterior

5.11.3. Eliminar tu Calificación

1. Ve a tus calificaciones en el perfil
 2. Desliza el dedo sobre la calificación
 3. Toca **“Eliminar”**
-

5.12. Comentarios

5.12.1. Dejar un Comentario

5.13. Escribir

1. Abre el libro que quieres comentar
2. Desplázate a **“Comentarios”**
3. Toca **“Agregar Comentario”**
4. Escribe tu opinión

5.14. Publicar

5. Revisa tu comentario
6. Toca **“Publicar”**
7. Tu comentario aparecerá en la lista

5.15. Consejos

Comentarios Útiles

- Sé respetuoso con otros usuarios
- Evita spoilers importantes
- Comparte tu opinión honesta
- Ayuda a otros lectores

5.15.1. Ver Comentarios

Comentarios

Usuario123 · Hace 2 días

"¡Excelente libro! Lo recomiendo totalmente para amantes de la ciencia ficción."

Lector_Ávido · Hace 1 semana

"Muy buena narrativa, aunque el final fue un poco predecible."

5.15.2. Editar tu Comentario

1. Encuentra tu comentario
2. Toca el **icono de editar**
3. Modifica el texto
4. Toca **“Guardar”**

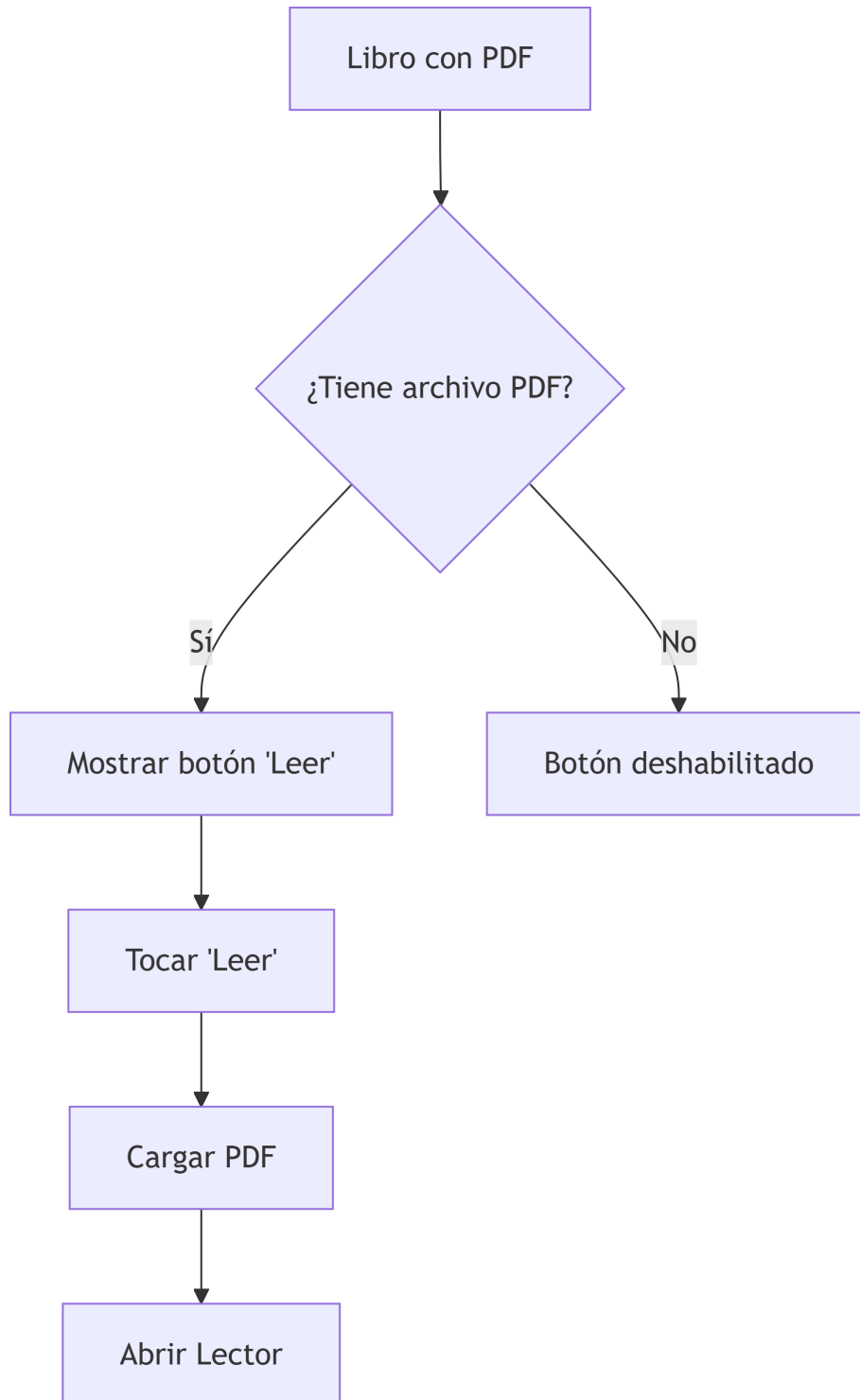
5.15.3. Eliminar tu Comentario

1. Encuentra tu comentario
2. Toca el **icono de eliminar**
3. Confirma la eliminación

5. *Manual de Usuario de Libris*

5.16. Lector de PDF

5.16.1. Abrir un PDF



1. **Selecciona un libro** que tenga archivo PDF
2. **Toca el botón “Leer”** en la parte inferior
3. El lector de PDF se abrirá automáticamente

5.16.2. Navegar en el PDF

5.16.3. Controles Básicos

- **Deslizar hacia arriba/abajo:** Navegar páginas
- **Pellizcar:** Hacer zoom
- **Tocar:** Mostrar/ocultar controles

5.16.4. Funciones Avanzadas

- **Barra de progreso:** Ver número de página
- **Botón retroceder:** Volver a detalles
- **Rotación:** Girar pantalla automáticamente

Formatos Soportados

Actualmente Libris soporta archivos **PDF** exclusivamente. Otros formatos estarán disponibles en futuras versiones.

5.17. Tu Perfil

5.17.1. Ver Información de Perfil

En la pantalla de perfil encontrarás:

Sección	Contenido
Datos Personales	Nombre, apellido, email, usuario
Estadísticas	Libros leídos, en progreso, pendientes
Mis Lecturas	Lista de todos tus libros registrados
Mis Calificaciones	Todas tus valoraciones
Mis Comentarios	Todos tus comentarios publicados

5.17.2. Editar Perfil

Función en Desarrollo

La edición de perfil estará disponible en próximas versiones. Por ahora, contacta al administrador para cambios.

5.17.3. Cerrar Sesión

1. Ve a tu **Perfil**
 2. Desplázate hasta abajo
 3. Toca “**Cerrar Sesión**”
 4. Confirma la acción
-

5.18. Consejos y Trucos

5.18.1. Maximiza tu Experiencia

5.19. Organización

Mantén tus lecturas organizadas: - Actualiza el estado regularmente - Usa calificaciones para recordar buenos libros - Agrega comentarios con tus impresiones

5.20. Exploración

Descubre nuevo contenido: - Revisa todas las categorías - Lee comentarios de otros usuarios - Mira las calificaciones promedio

5.21. Lectura

Disfruta la lectura: - Usa el modo horizontal para PDFs - Ajusta el brillo de tu pantalla - Lee en un lugar cómodo y con buena luz

5.22. Comunidad

Participa en la comunidad: - Deja calificaciones honestas - Escribe comentarios útiles
- Respeta las opiniones de otros

5.23. Solución de Problemas Comunes

5.23.1. No puedo iniciar sesión

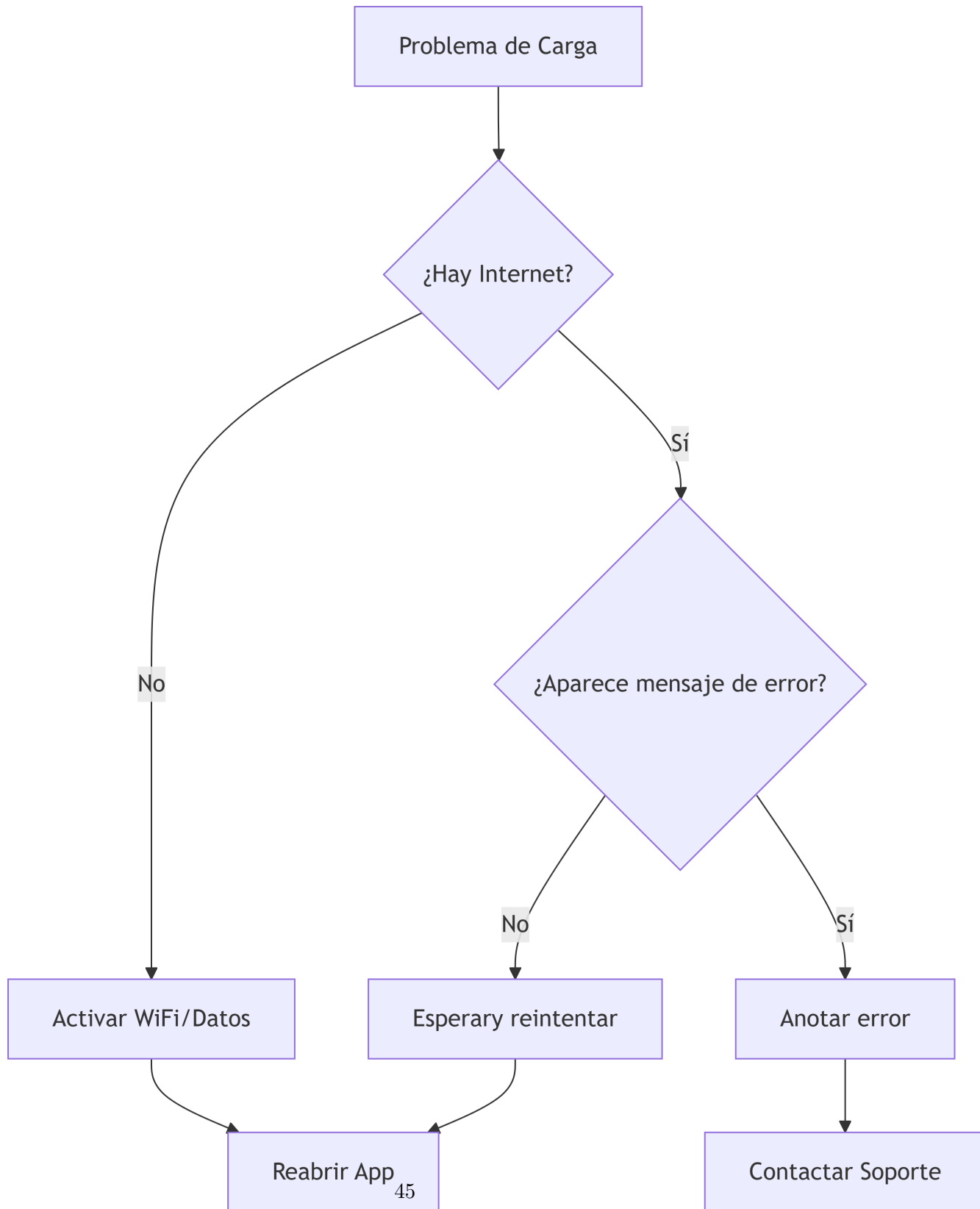
Verificaciones

1. **Usuario correcto:** Verifica que no haya espacios extra
2. **Contraseña correcta:** Distingue mayúsculas y minúsculas
3. **Conexión a Internet:** Revisa tu WiFi o datos móviles
4. **Cuenta activa:** Asegúrate de estar registrado

Solución: - Intenta cerrar y abrir la app - Verifica tu conexión - Contacta al soporte si persiste

5. *Manual de Usuario de Libris*

5.23.2. Los libros no cargan



Causas comunes: - Conexión lenta o inestable - Servidor temporalmente no disponible
- Error en la aplicación

Solución: 1. Verifica tu conexión a Internet 2. Cierra y vuelve a abrir la app 3. Espera unos minutos y reintenta 4. Si persiste, contacta al soporte

5.23.3. El PDF no se abre

Posibles causas: - El libro no tiene archivo PDF asociado - Archivo PDF corrupto - Problema de conexión durante la descarga

Solución: 1. Verifica que el botón “Leer” esté habilitado 2. Asegura buena conexión a Internet 3. Intenta con otro libro 4. Reporta el problema si es recurrente

5.23.4. La app se cierra inesperadamente

! Pasos para Resolver

1. **Actualiza la app** a la última versión
2. **Reinicia tu dispositivo**
3. **Libera espacio** en tu teléfono
4. **Reinstala la app** si persiste
5. **Contacta soporte** con detalles del error

5.24. Preguntas Frecuentes (FAQ)

5.24.1. General

¿Libris es gratis? Sí, Libris es completamente gratuito para todos los usuarios.

¿Necesito Internet para usar Libris? Sí, necesitas conexión a Internet para acceder al catálogo y sincronizar tus datos.

¿Mis datos están seguros? Sí, usamos autenticación JWT y almacenamiento seguro para proteger tu información.

5.24.2. Lecturas

¿Puedo leer sin descargar? Los PDFs se cargan en streaming, no necesitas descargarlos completamente.

¿Cuántos libros puedo agregar a mis lecturas? No hay límite, puedes agregar tantos libros como desees.

¿Puedo leer sin conexión? Actualmente no, se requiere conexión para cargar los PDFs. Esta función está planificada para futuras versiones.

5.24.3. Calificaciones y Comentarios

¿Puedo calificar sin comentar? Sí, las calificaciones y comentarios son independientes.

¿Otros usuarios ven mis calificaciones? Sí, las calificaciones son públicas y contribuyen al promedio del libro.

¿Puedo editar mi comentario? Sí, puedes editar o eliminar tus comentarios en cualquier momento.

5.24.4. Cuenta

¿Puedo cambiar mi nombre de usuario? Contacta al administrador para cambios de cuenta.

¿Cómo elimino mi cuenta? Contacta al soporte técnico para solicitar la eliminación de cuenta.

¿Puedo usar Libris en varios dispositivos? Sí, inicia sesión con tus credenciales en cualquier dispositivo.

5.25. Conclusión

¡Felicidades! Ahora conoces todas las funciones de Libris.

i Recuerda

- Explora el catálogo completo
- Mantén actualizado tu progreso
- Participa con calificaciones y comentarios
- Disfruta tu experiencia de lectura

¿Tienes dudas? Consulta las [Preguntas Frecuentes](#)

¡Feliz lectura!

6. Preguntas Frecuentes

FAQ - Respuestas a las preguntas más comunes

7. Preguntas Frecuentes (FAQ)

7.1. Sobre la Aplicación

7.1.1. ¿Qué es Libris?

Libris es una aplicación móvil para gestionar tu biblioteca personal de lectura. Te permite explorar un catálogo de libros, mangas, novelas y material educativo, registrar tu progreso de lectura, calificar obras y compartir comentarios con otros lectores.

7.1.2. ¿Libris es gratuito?

Sí, **Libris es completamente gratuito** para todos los usuarios. No hay costos ocultos, suscripciones ni compras dentro de la aplicación.

7.1.3. ¿En qué plataformas está disponible Libris?

Actualmente Libris está disponible para:

- **Android** (5.0 Lollipop o superior)
- **iOS** (11.0 o superior)
- **Web** (en desarrollo)

7.1.4. ¿Necesito crear una cuenta para usar Libris?

Sí, necesitas crear una cuenta para usar Libris. Esto permite:

- Guardar tu progreso de lectura
- Sincronizar tus datos entre dispositivos
- Participar con calificaciones y comentarios
- Personalizar tu experiencia

La creación de cuenta es rápida y solo requiere usuario, email y contraseña.

7.2. Cuenta y Seguridad

7.2.1. ¿Cómo creo una cuenta?

1. Abre la aplicación Libris
2. Toca “**Registrarse**”
3. Completa el formulario con:
 - Usuario (único)
 - Email (válido)
 - Contraseña (mínimo 8 caracteres)
 - Nombre y apellido
4. Toca “**Registrar**”

Ver detalles en el [Manual de Usuario](#).

7.2.2. ¿Olvidé mi contraseña, qué hago?

Actualmente debes **contactar al administrador** del sistema para restablecer tu contraseña.

Note

La función de “Olvidé mi contraseña” estará disponible en futuras versiones.

Contacto: soporte@libris.app

7.2.3. ¿Mis datos están seguros?

Sí, Libris utiliza medidas de seguridad robustas:

- **Autenticación JWT** - Tokens seguros de sesión
- **Almacenamiento encriptado** - Tus credenciales están protegidas
- **HTTPS** - Todas las comunicaciones están cifradas
- **Sin rastreo** - No recopilamos datos innecesarios

7.2.4. ¿Puedo usar la misma cuenta en múltiples dispositivos?

Sí, puedes iniciar sesión con tu cuenta en cualquier dispositivo compatible. Tus datos se sincronizarán automáticamente.

7. Preguntas Frecuentes (FAQ)

Tip

Cierra sesión en dispositivos que ya no uses para mantener tu cuenta segura.

7.2.5. ¿Cómo elimino mi cuenta?

Para eliminar tu cuenta permanentemente:

1. Contacta a **soporte@libris.app**
2. Proporciona tu nombre de usuario
3. Confirma la solicitud de eliminación

Acción Irreversible

La eliminación de cuenta es permanente y eliminará: - Tu perfil - Todas tus lecturas registradas - Tus calificaciones y comentarios - Cualquier dato asociado a tu cuenta

7.3. Catálogo y Contenido

7.3.1. ¿Cuántos libros hay en el catálogo?

El catálogo de Libris incluye:

- Libros (ficción y no ficción)
- Mangas
- Novelas
- Material educativo

El número exacto varía ya que el catálogo se actualiza regularmente.

7.3.2. ¿Puedo agregar mis propios libros?

Actualmente **no**, solo los administradores pueden agregar contenido al catálogo. Esta función está planificada para futuras versiones.

7. Preguntas Frecuentes (FAQ)

7.3.3. ¿Los libros son completos o solo muestras?

Depende del contenido:

- **Libros con PDF:** Acceso completo al documento
- **Libros sin PDF:** Solo información y metadatos
- **Material educativo:** Generalmente completo

7.3.4. ¿Puedo descargar libros para leer sin conexión?

Actualmente no, necesitas conexión a Internet para:

- Cargar el catálogo
- Abrir PDFs en el lector
- Sincronizar tu progreso

i Próximamente

El modo offline está en desarrollo y permitirá descargar libros para lectura sin conexión.

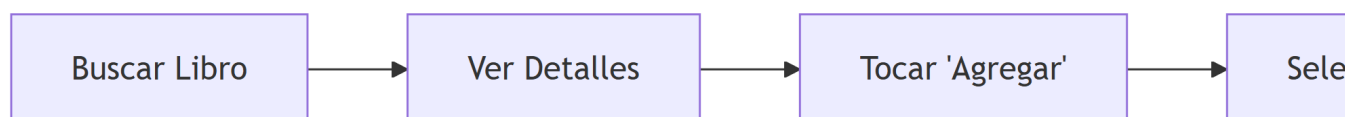
7.3.5. ¿Cómo sé si un libro tiene PDF disponible?

En la página de detalles del libro:

- **Botón “Leer” habilitado** = Tiene PDF
- **Botón “Leer” deshabilitado** = Sin PDF

7.4. Lectura y Progreso

7.4.1. ¿Cómo agrego un libro a mis lecturas?



1. Busca el libro en el catálogo
2. Abre los detalles del libro

7. Preguntas Frecuentes (FAQ)

3. Toca “**Agregar a Mis Lecturas**”
4. Selecciona el estado (Leyendo/Pendiente/Completado)

Ver más en [Manual de Usuario](#).

7.4.2. ¿Puedo cambiar el estado de una lectura?

Sí, puedes actualizar el estado en cualquier momento:

1. Ve a **Perfil → Mis Lecturas**
2. Selecciona el libro
3. Cambia entre: Leyendo Pendiente Completado

7.4.3. ¿Se guarda automáticamente mi progreso?

Sí, el estado de tus lecturas se guarda automáticamente en el servidor cuando:

- Agregas un libro a tus lecturas
- Cambias el estado de una lectura
- Actualizas cualquier información

7.4.4. ¿Libris recuerda en qué página me quedé?

Actualmente no, el lector de PDF no guarda la posición específica de lectura. Esta función está planificada para próximas versiones.

Consejo

Usa marcadores mentales o anota el número de página externamente hasta que esta función esté disponible.

7.4.5. ¿Puedo eliminar un libro de mis lecturas?

Sí, para eliminar:

1. Ve a **Perfil → Mis Lecturas**
2. Encuentra el libro
3. Desliza hacia la izquierda
4. Toca “**Eliminar**”
5. Confirma la acción

7.5. Calificaciones y Comentarios

7.5.1. ¿Cómo califico un libro?

1. Abre los detalles del libro
2. Desplázate a la sección “**Calificaciones**”
3. Toca las estrellas para seleccionar tu calificación (1-5)
4. La calificación se guarda automáticamente

7.5.2. ¿Puedo cambiar mi calificación?

Sí, simplemente vuelve al libro y toca un número diferente de estrellas. Tu calificación anterior será reemplazada.

7.5.3. ¿Puedo calificar sin haber leído el libro?

Técnicamente **sí**, la app no lo impide, pero te recomendamos:

! Calificaciones Honestas

Calificas solo libros que hayas leído para que las calificaciones sean útiles para otros usuarios.

7.5.4. ¿Cómo elimino mi calificación?

1. Ve a **Perfil** → **Mis Calificaciones**
2. Encuentra la calificación
3. Desliza hacia la izquierda
4. Toca “**Eliminar**”

7.5.5. ¿Cómo dejo un comentario?

1. Abre el libro
2. Desplázate a “**Comentarios**”
3. Toca “**Agregar Comentario**”
4. Escribe tu opinión
5. Toca “**Publicar**”

7. Preguntas Frecuentes (FAQ)

7.5.6. ¿Puedo editar mi comentario después de publicarlo?

Sí, puedes editar tus comentarios:

1. Encuentra tu comentario en el libro
2. Toca el icono de **editar**
3. Modifica el texto
4. Toca **“Guardar”**

7.5.7. ¿Los comentarios tienen límite de caracteres?

Sí, los comentarios tienen un **límite de caracteres** definido por el servidor (generalmente 500-1000 caracteres).

7.5.8. ¿Puedo ver quién calificó o comentó?

Sí, cada calificación y comentario muestra:

- Nombre de usuario del autor
- Fecha de publicación
- Calificación (si corresponde)

7.5.9. ¿Puedo reportar comentarios inapropiados?

Actualmente **no** hay función de reporte. Si encuentras contenido inapropiado, contacta a **soporte@libris.app** con:

- Nombre del libro
- Usuario del comentario
- Razón del reporte

7.6. Lector de PDF

7.6.1. ¿Qué formatos de archivo soporta Libris?

Actualmente solo soporta **PDF (Portable Document Format)**.

Próximos formatos

Soporte para EPUB, MOBI y otros formatos está en evaluación para futuras versiones.

7.6.2. ¿Cómo abro el lector de PDF?

1. Busca un libro que tenga PDF disponible
2. Abre los detalles del libro
3. Toca el botón “**Leer**”
4. El lector se abrirá automáticamente

7.6.3. ¿Puedo hacer zoom en el PDF?

Sí, usa gestos táctiles estándar:

- **Pellizcar** con dos dedos = Zoom in/out
- **Doble toque** = Ajustar al ancho de página
- **Arrastrar** = Mover la página cuando está ampliada

7.6.4. ¿Cómo navego entre páginas?

- **Desliza hacia arriba** = Siguiente página
- **Desliza hacia abajo** = Página anterior
- **Barra de progreso** = Ir a página específica

7.6.5. ¿El lector tiene modo nocturno?

Actualmente no, el lector muestra el PDF en su formato original. El modo nocturno está planificado para futuras versiones.

7.6.6. ¿Puedo rotar la pantalla al leer?

Sí, el lector soporta rotación automática:

- **Vertical** (portrait) - Mejor para lectura normal
- **Horizontal** (landscape) - Mejor para documentos anchos

7. Preguntas Frecuentes (FAQ)

7.6.7. ¿Puedo compartir el PDF?

No, por razones de copyright y seguridad, no es posible exportar o compartir los PDFs desde la app.

7.6.8. El PDF no carga, ¿qué hago?

Posibles soluciones:

1. **Verifica tu conexión** a Internet
2. **Cierra y vuelve a abrir** el lector
3. **Reinicia la app**
4. **Verifica espacio** en tu dispositivo
5. **Intenta con otro libro** para ver si es un problema general

Si persiste, reporta el problema a soporte.

7.7. Conexión y Sincronización

7.7.1. ¿Necesito Internet para usar Libris?

Sí, actualmente Libris requiere conexión a Internet para:

- Cargar el catálogo de libros
- Sincronizar tus lecturas
- Abrir PDFs
- Guardar calificaciones y comentarios
- Iniciar sesión

7.7.2. ¿Funciona con WiFi y datos móviles?

Sí, Libris funciona con cualquier tipo de conexión a Internet:

- WiFi
- Datos móviles (3G/4G/5G)

Consejo de Ahorro

Los PDFs pueden consumir datos. Si tienes plan limitado, usa WiFi cuando sea posible.

7. Preguntas Frecuentes (FAQ)

7.7.3. ¿Qué pasa si pierdo la conexión mientras leo?

Si la conexión se pierde:

- **Páginas ya cargadas** seguirán visibles
- **Páginas nuevas** no cargarán
- **Cambios de estado** no se guardarán hasta reconectar

7.7.4. ¿Mis datos se sincronizan entre dispositivos?

Sí, cuando usas la misma cuenta en múltiples dispositivos:

- Lecturas agregadas
- Estados de lectura
- Calificaciones
- Comentarios

Todo se sincroniza automáticamente cuando hay conexión.

7.7.5. ¿Con qué frecuencia se sincroniza?

La sincronización es **inmediata** cuando:

- Agregas una lectura
- Cambias un estado
- Publicas una calificación
- Dejas un comentario

7.8. Configuración y Personalización

7.8.1. ¿Puedo cambiar el tema (claro/oscuro)?

Actualmente no, Libris usa el tema predeterminado. Los temas personalizables están planificados para futuras versiones.

7.8.2. ¿Puedo cambiar el tamaño de fuente?

No en la versión actual. Esta función está en la hoja de ruta para próximas actualizaciones.

7. Preguntas Frecuentes (FAQ)

7.8.3. ¿Puedo cambiar mi nombre de usuario?

No directamente en la app. Para cambios de cuenta, contacta a **soporte@libris.app**.

7.8.4. ¿Puedo cambiar mi email?

Similar al nombre de usuario, los cambios de email deben solicitarse al soporte.

7.8.5. ¿Dónde están las notificaciones?

Las notificaciones **no están implementadas** en la versión actual. Próximas versiones incluirán:

- Recordatorios de lectura
 - Nuevos libros en el catálogo
 - Respuestas a tus comentarios
-

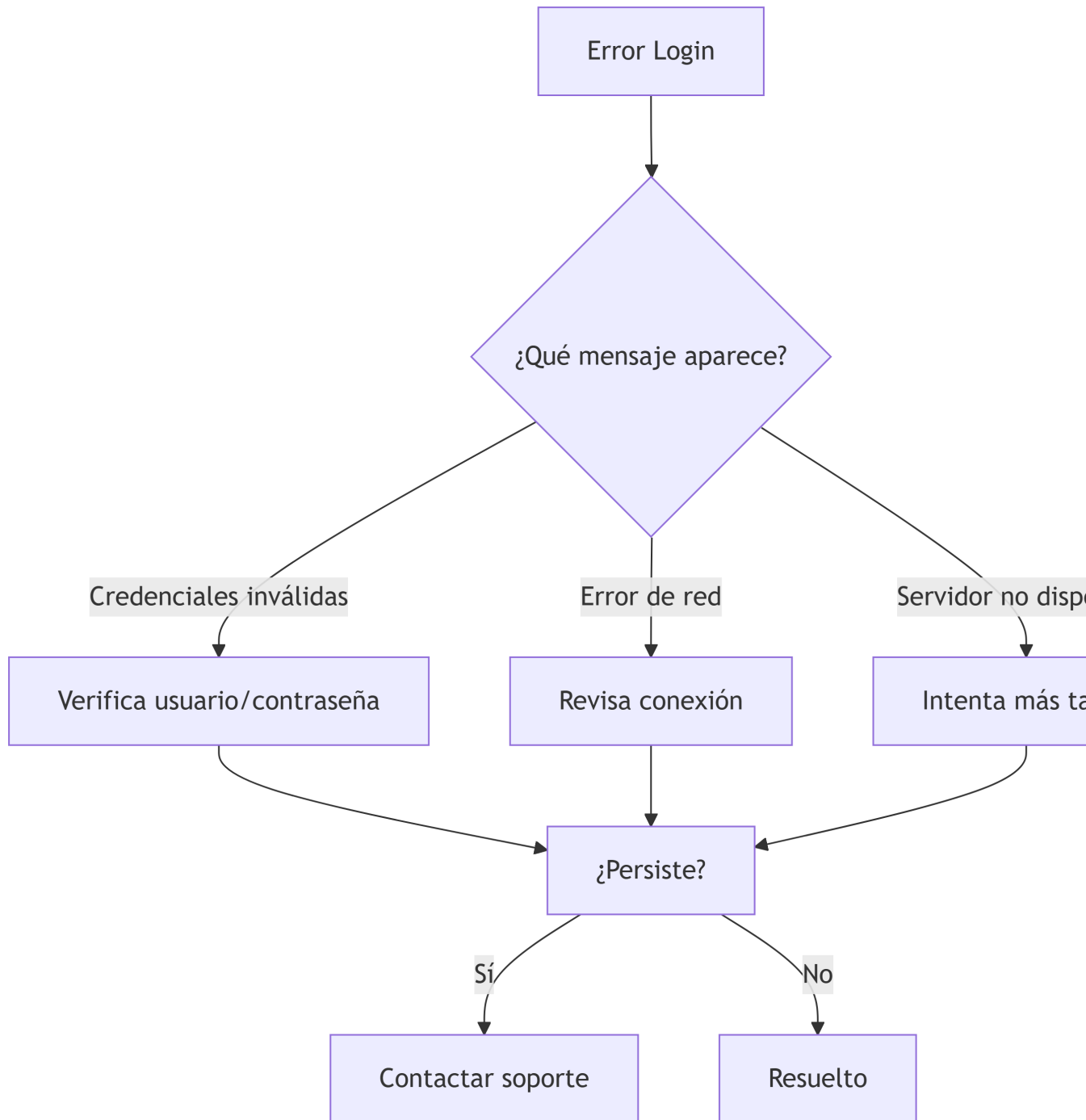
7.9. Problemas Técnicos

7.9.1. La app se cierra inesperadamente

Soluciones:

1. **Actualiza la app** a la última versión
2. **Reinicia tu dispositivo**
3. **Libera espacio** de almacenamiento
4. **Reinstala la app** (perderás datos locales)
5. **Reporta el error** a soporte con:
 - Modelo de dispositivo
 - Sistema operativo y versión
 - Qué estabas haciendo cuando ocurrió

7.9.2. No puedo iniciar sesión



Verificaciones:

7. Preguntas Frecuentes (FAQ)

- Usuario correcto (sin espacios)
- Contraseña correcta (case-sensitive)
- Conexión a Internet activa
- Servidor operativo

7.9.3. Los libros no cargan o aparecen en blanco

Causas posibles:

1. **Conexión lenta** - Espera más tiempo
2. **Servidor sobrecargado** - Intenta más tarde
3. **Cache corrupto** - Limpia caché de la app
4. **Error de la app** - Reinstala

Pasos:

1. Verifica WiFi/datos
2. Cierra la app completamente
3. Limpia caché (Configuración → Apps → Libris → Borrar caché)
4. Abre la app nuevamente
5. Si persiste, reinstala

7.9.4. El lector PDF está en blanco o no carga

Soluciones:

1. Verifica tu conexión a Internet
2. Cierra y vuelve a abrir el PDF
3. Reinicia la app
4. Intenta con otro libro
5. Verifica espacio en dispositivo
6. Actualiza la app

7.9.5. Mis calificaciones o comentarios no se guardan

Razones:

- Perdiste la conexión antes de guardar
- Error del servidor
- Sesión expirada

Solución:

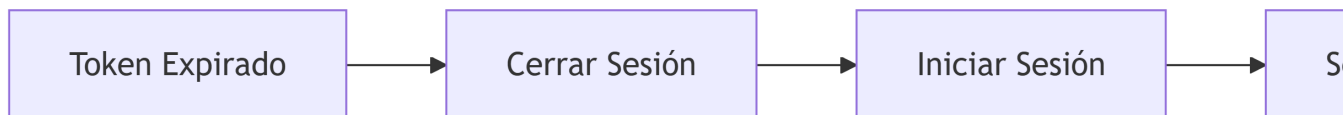
7. Preguntas Frecuentes (FAQ)

1. Verifica que tienes conexión
2. Cierra sesión y vuelve a iniciar
3. Intenta guardar nuevamente
4. Si falla repetidamente, reporta a soporte

7.9.6. Errores de “Token expirado” o “No autorizado”

Este error significa que tu sesión ha expirado.

Solución:



1. **Cierra sesión** manualmente
2. **Inicia sesión** nuevamente
3. Tu sesión se renovará

i Note

Las sesiones expiran después de cierto tiempo por seguridad (generalmente 1 hora de inactividad).

7.10. Rendimiento y Datos

7.10.1. ¿Cuánto espacio ocupa Libris?

- **Instalación inicial:** ~50-100 MB
- **Con uso:** Variable según caché

7.10.2. ¿Cuántos datos consume Libris?

Depende del uso:

Actividad	Consumo Aproximado
Navegar catálogo	~1-5 MB/sesión
Leer un PDF (100 pgs)	~5-20 MB

7. Preguntas Frecuentes (FAQ)

Actividad	Consumo Aproximado
Calificar/comentar	<1 MB

Ahorra Datos

Usa WiFi para leer PDFs extensos y evitar consumo excesivo de datos móviles.

7.10.3. ¿Libris almacena datos en caché?

Sí, Libris guarda temporalmente:

- Portadas de libros
- Metadatos del catálogo
- Algunas páginas de PDFs

Esto mejora la velocidad pero **no** permite uso offline.

7.10.4. ¿Cómo limpio la caché?

Android: 1. Ve a Configuración → Apps 2. Busca Libris 3. Toca “Almacenamiento” 4. Toca “Borrar caché”

iOS: 1. Generalmente necesitas desinstalar y reinstalar 2. O espera a que iOS limpie caché automáticamente

7.11. Actualizaciones

7.11.1. ¿Cómo sé si hay una actualización disponible?

- **Android:** Google Play Store te notificará
- **iOS:** App Store te notificará
- Puedes verificar manualmente en la tienda

7. Preguntas Frecuentes (FAQ)

7.11.2. ¿Las actualizaciones son automáticas?

Depende de la configuración de tu dispositivo:

- Si tienes **actualizaciones automáticas activadas**, sí
- Si no, debes actualizar manualmente

7.11.3. ¿Perderé mis datos al actualizar?

No, tus datos están a salvo porque:

- Se almacenan en el servidor (lecturas, calificaciones, comentarios)
- Las actualizaciones solo cambian el código de la app
- Tu sesión se mantendrá (o simplemente vuelve a iniciar sesión)

7.11.4. ¿Con qué frecuencia se actualiza Libris?

El calendario de actualizaciones varía, pero típicamente:

- **Versiones mayores** (1.0 → 2.0): Cada 6-12 meses
- **Versiones menores** (1.0 → 1.1): Cada 2-3 meses
- **Parches** (1.0.0 → 1.0.1): Según necesidad

7.12. Soporte y Ayuda

7.12.1. ¿Cómo contacto al soporte?

Canal	Disponibilidad	Tiempo de Respuesta
Email	soporte@libris.app	24-48 horas
Chat en App	Lunes-Viernes 9-18h	Inmediato
GitHub Issues	24/7	Variable

7.12.2. ¿Qué información debo proporcionar al reportar un problema?

Para ayudarte mejor, incluye:

7. Preguntas Frecuentes (FAQ)

Versión de la app (ve a Perfil → Acerca de)
Modelo del dispositivo (ej: iPhone 12, Samsung Galaxy S21)
Sistema operativo y versión (ej: iOS 15.1, Android 12)
Descripción clara del problema
Pasos para reproducir el error
Capturas de pantalla (si es posible)
Mensajes de error exactos

7.12.3. ¿Puedo solicitar nuevas funciones?

¡Sí! Nos encanta recibir sugerencias:

1. Envía un email a **features@libris.app**
2. O abre un Issue en GitHub
3. Describe claramente tu idea
4. Explica por qué sería útil

7.12.4. ¿Cómo puedo contribuir al proyecto?

Si eres desarrollador:

1. Visita el repositorio en GitHub
2. Lee la [Guía de Contribución](#)
3. Haz un fork del proyecto
4. Envía Pull Requests

Si no eres desarrollador:

- Reporta bugs
 - Sugiere funciones
 - Ayuda con traducciones
 - Comparte la app
-

7.13. Futuras Funciones

7.13.1. ¿Qué funciones están planificadas?

Roadmap 2024-2025

Próximo Release (Q2 2024): - Modo offline - Tema oscuro - Notificaciones

Mediano Plazo (Q3-Q4 2024): - Sistema de amigos - Estadísticas avanzadas
- Logros y badges - Listas personalizadas

Largo Plazo (2025): - Audiolibros - Versión web completa - Recomendaciones con IA - Chat entre usuarios

7.13.2. ¿Cuándo estará disponible el modo offline?

El modo offline es nuestra **prioridad #1** y está previsto para **Q2 2024**.

7.13.3. ¿Habrá soporte para otros formatos (EPUB, MOBI)?

Sí, está en evaluación para **Q4 2024**. EPUB será el próximo formato soportado.

7.13.4. ¿Libris estará disponible en web?

Sí, una versión web básica está en desarrollo y se lanzará en **Q3 2024**.

7.14. Más Información

Recursos Adicionales

- [Manual de Usuario Completo](#)
- [Guía de Instalación](#)
- [Documentación Técnica](#)
- [API Reference](#)
- [Troubleshooting](#)

7. Preguntas Frecuentes (FAQ)

¿No encontraste tu pregunta?

Contacta a **soporte@libris.app** o consulta la documentación completa.

Última actualización: Febrero 2026

Versión de la app: 1.0.0

Part II.

Documentacion Tecnica

8. Documentación Técnica

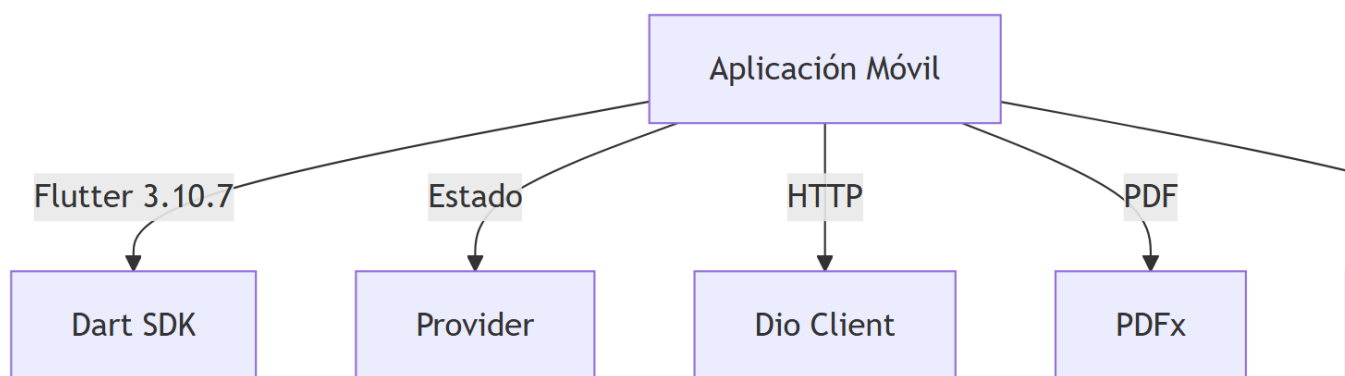
Arquitectura y estructura del proyecto Libris

9. Documentación Técnica

9.1. Descripción General del Proyecto

Libris es una aplicación móvil multiplataforma desarrollada en **Flutter 3.10.7** que permite a los usuarios gestionar su biblioteca personal de lectura, consumiendo una API REST backend basada en Django.

9.1.1. Stack Tecnológico

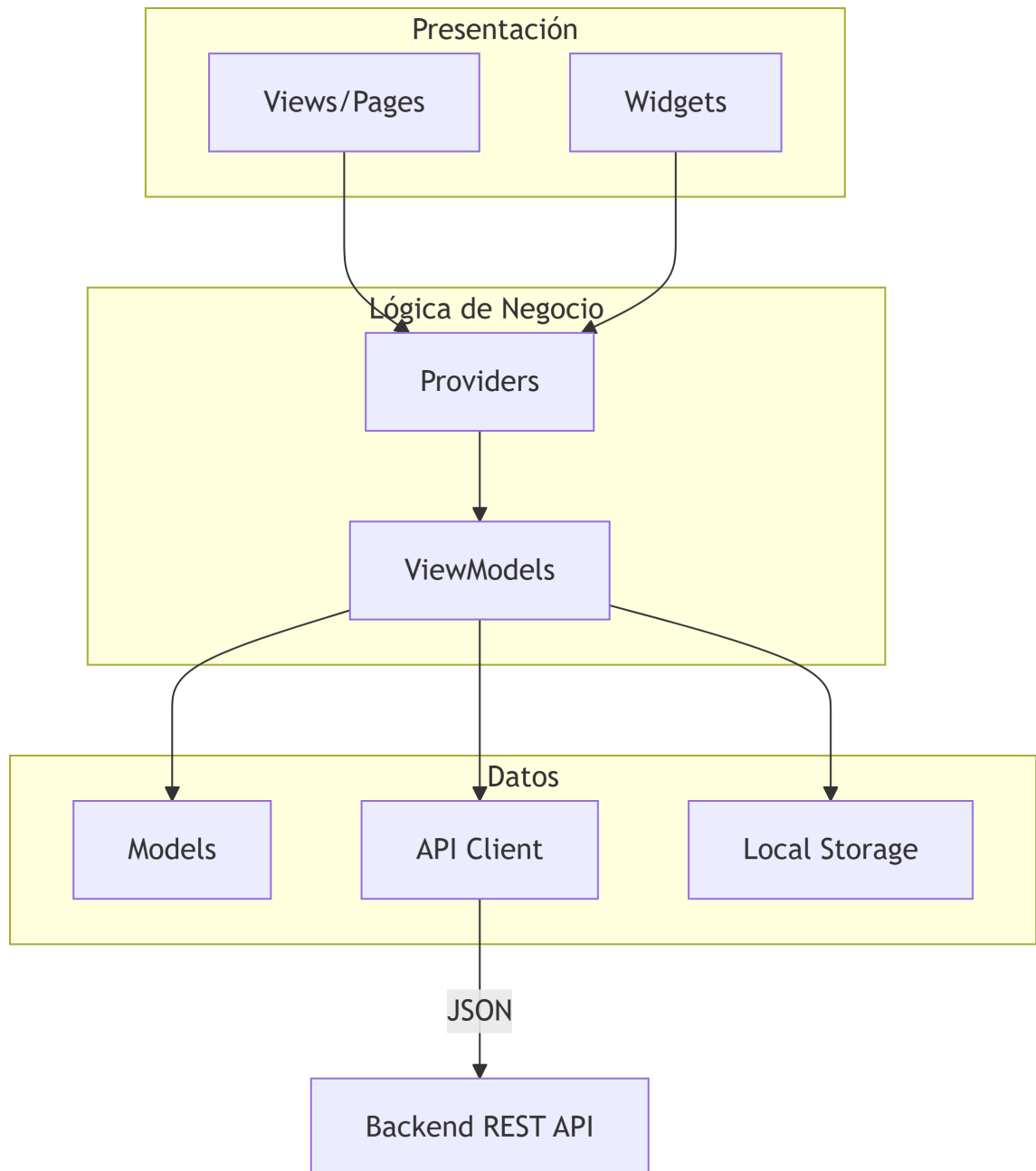


Componente	Tecnología	Versión
Framework	Flutter	3.10.7
Lenguaje	Dart	^3.10.7
Gestión de Estado	Provider	6.1.5+1
Cliente HTTP	Dio	5.4.3+1
Almacenamiento Seguro	flutter_secure_storage	9.0.0
Visor PDF	PDFx	2.9.2
Fuentes	Google Fonts	6.2.0
Backend	Django REST Framework	-
Autenticación	JWT	-

9.2. Arquitectura de la Aplicación

9.2.1. Patrón de Arquitectura

Libris utiliza una arquitectura **MVVM (Model-View-ViewModel)** con **Provider** para la gestión de estado:



9.2.2. Capas de la Aplicación

9.2.2.1. 1. Capa de Presentación (UI)

Ubicación: `lib/pages/`

- `home_page.dart`: Pantalla principal con categorías
- `register_page.dart`: Registro de nuevos usuarios
- `profile_page.dart`: Perfil del usuario
- `material_detail_page.dart`: Detalles de un item
- `pdf_reader_page.dart`: Lector de PDF
- `admin_panel_page.dart`: Panel administrativo

9.2.2.2. 2. Capa de Lógica (Business Logic)

Ubicación: `lib/providers/` (implícito en la estructura)

- Gestión de estado con Provider
- Lógica de autenticación
- Manejo de datos del usuario
- Control de flujo de la aplicación

9.2.2.3. 3. Capa de Datos (Data)

Ubicación: `lib/api/` y `lib/models/`

- **ApiClient**: Cliente HTTP configurado
- **Models**: Clases de datos (User, Book, Rating, etc.)
- **Repositories**: Abstracción de fuentes de datos

9.3. Estructura del Proyecto

```
libris/  
  lib/  
    main.dart          # Punto de entrada  
    api/  
      api_client.dart  # Cliente Dio configurado  
      auth_service.dart # Servicios de autenticación  
      book_service.dart # Servicios de libros
```

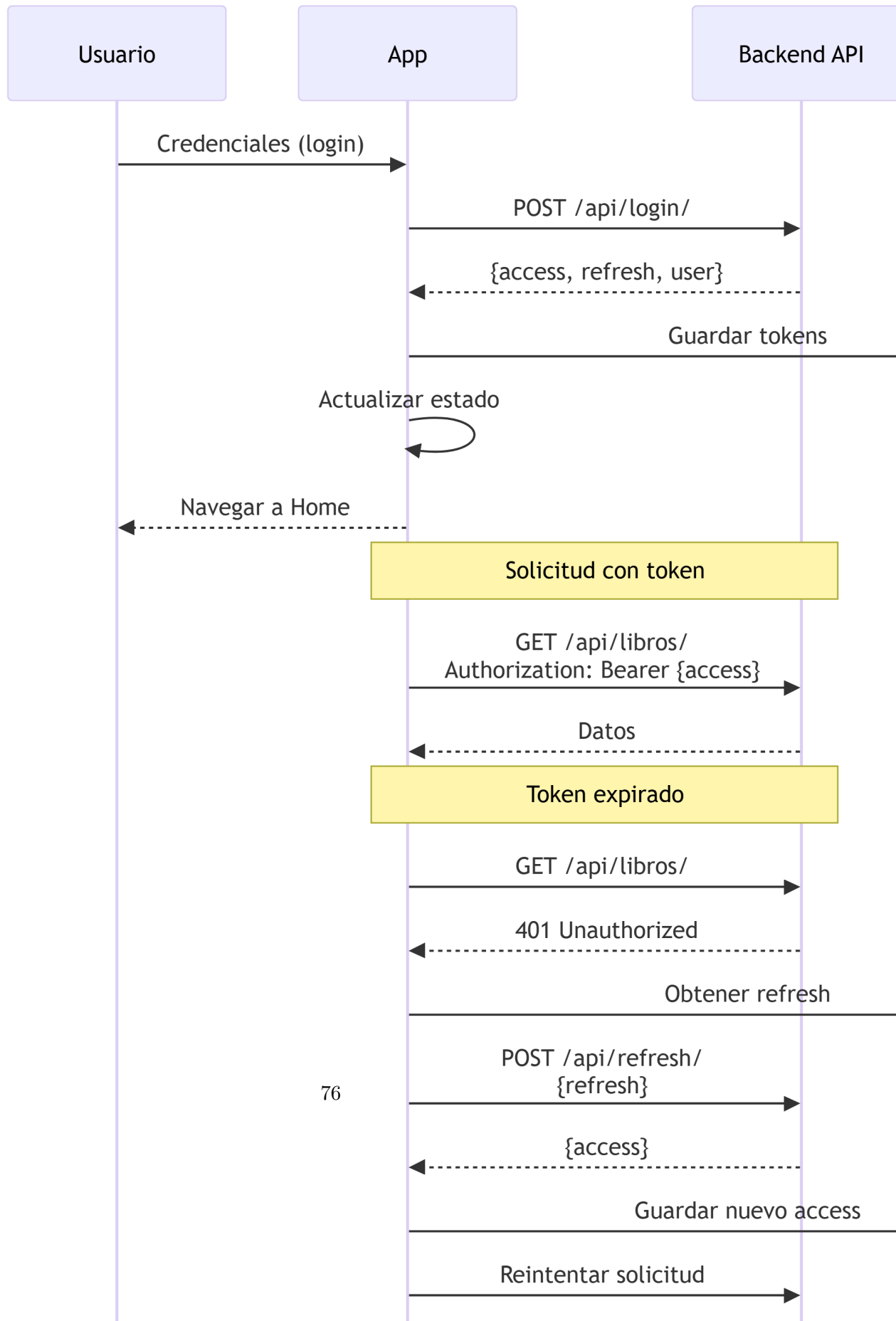
9. Documentación Técnica

```
...                                # Otros servicios
models/
  user.dart                        # Modelo de Usuario
  book.dart                       # Modelo de Libro
  reading_record.dart             # Modelo de Registro
  rating.dart                    # Modelo de Calificación
  comment.dart                   # Modelo de Comentario
pages/
  home_page.dart
  register_page.dart
  profile_page.dart
  material_detail_page.dart
  pdf_reader_page.dart
  admin_panel_page.dart
widgets/
  book_card.dart                 # Card de libro
  rating_stars.dart              # Estrellas de calificación
  ...                            # Otros widgets
providers/
  auth_provider.dart
  book_provider.dart
  user_provider.dart
theme/
  app_theme.dart                 # Tema de la app
  colors.dart                    # Colores personalizados
utils/
  constants.dart                 # Constantes globales
  validators.dart                # Validadores
  helpers.dart                   # Funciones auxiliares
android/                         # Configuración Android
ios/                             # Configuración iOS
web/                             # Configuración Web
test/                            # Tests unitarios
pubspec.yaml                     # Dependencias
analysis_options.yaml            # Reglas de análisis
```

9. Documentación Técnica

9.4. Flujo de Autenticación

9.4.1. Sistema JWT



9.4.2. Implementación

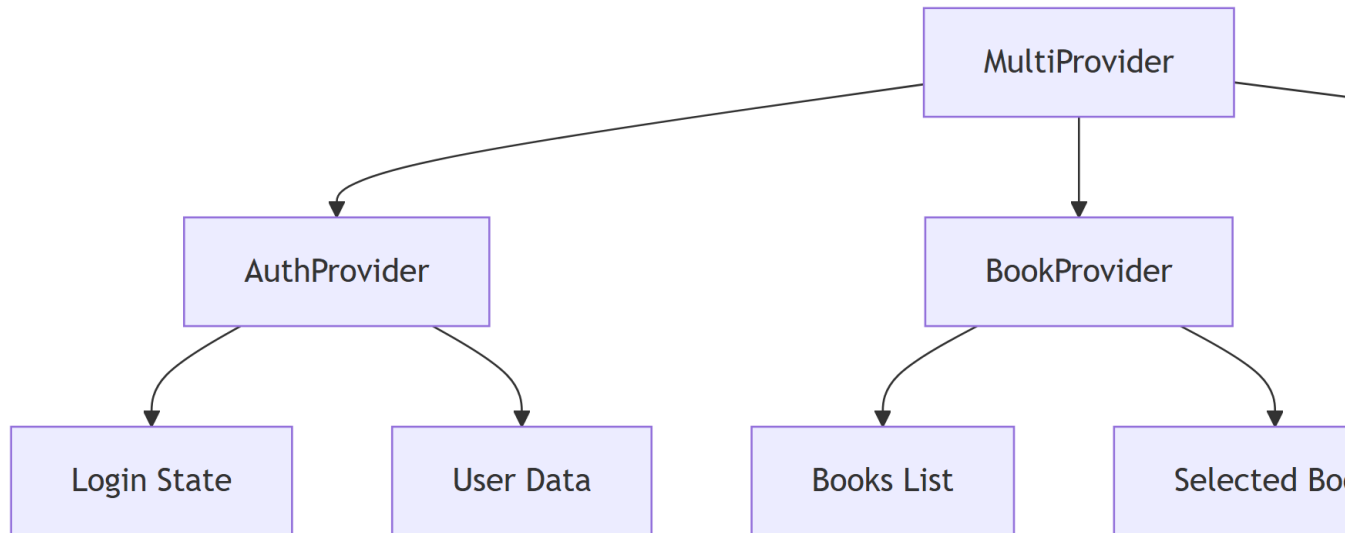
```
// Interceptor de autenticación
class AuthInterceptor extends Interceptor {
  final FlutterSecureStorage storage;

  @override
  Future<void> onRequest(
    RequestOptions options,
    RequestInterceptorHandler handler,
  ) async {
    final token = await storage.read(key: 'access_token');
    if (token != null) {
      options.headers['Authorization'] = 'Bearer $token';
    }
    handler.next(options);
  }

  @override
  Future<void> onError(
    DioException err,
    ErrorInterceptorHandler handler,
  ) async {
    if (err.response?.statusCode == 401) {
      // Intentar refresh del token
      final refreshed = await _refreshToken();
      if (refreshed) {
        // Reintentar la solicitud original
        final response = await _retry(err.requestOptions);
        handler.resolve(response);
        return;
      }
    }
    handler.next(err);
  }
}
```


9.5. Gestión de Estado

9.5.1. Provider Pattern



9.5.2. Ejemplo de Provider

```
class BookProvider extends ChangeNotifier {
  final ApiClient _apiClient;

  List<Book> _books = [];
  bool _isLoading = false;
  String? _error;

  List<Book> get books => _books;
  bool get isLoading => _isLoading;
  String? get error => _error;

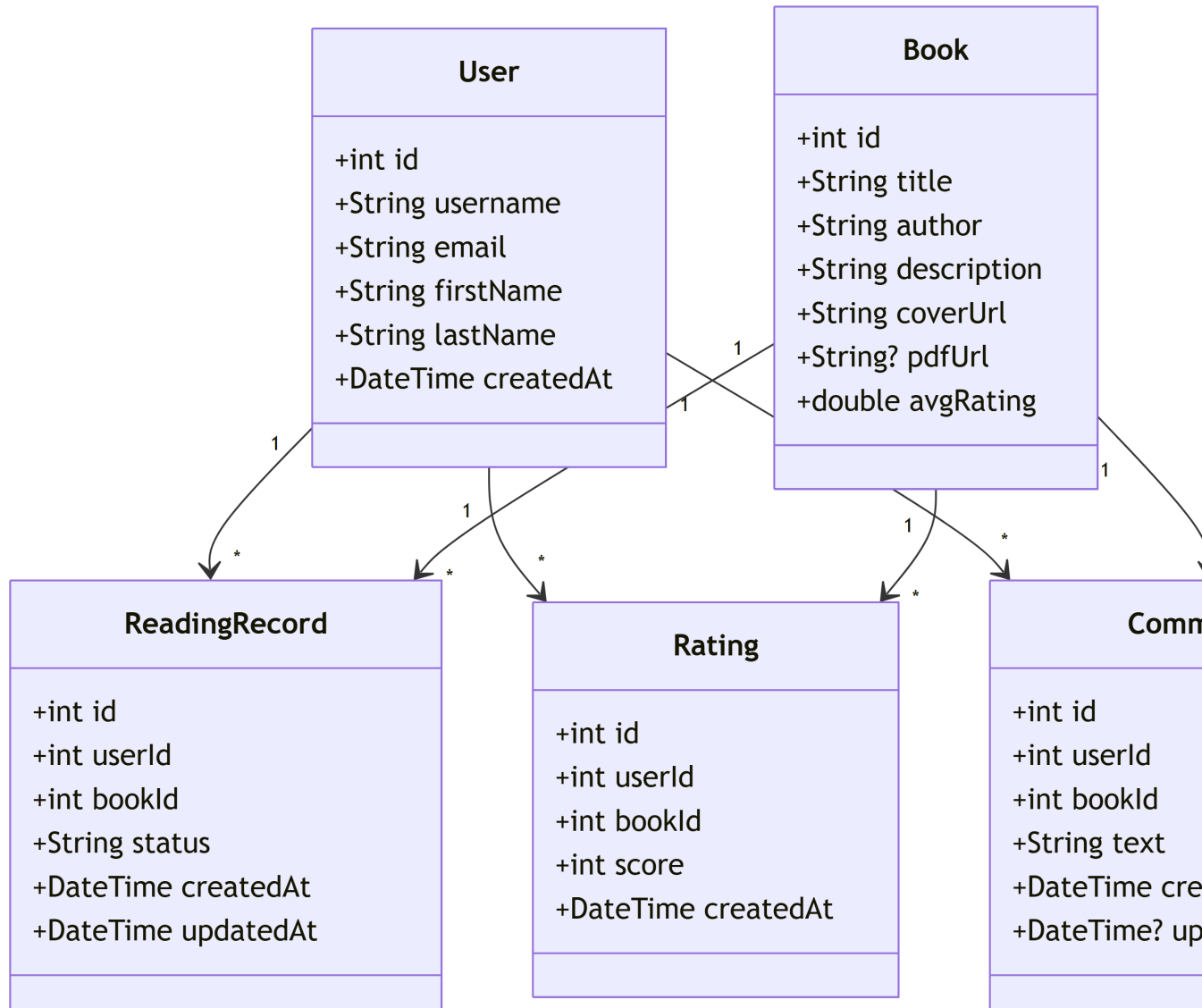
  Future<void> fetchBooks() async {
    _isLoading = true;
    _error = null;
    notifyListeners();

    try {
      final response = await _apiClient.get('/libros/');
      _books = (response.data as List)
    }
  }
}
```

```
        .map((json) => Book.fromJson(json))
        .toList();
    } catch (e) {
        _error = e.toString();
    } finally {
        _isLoading = false;
        notifyListeners();
    }
}
}
```

9.6. Modelos de Datos

9.6.1. Diagrama de Clases



9.6.2. Serialización JSON

Todos los modelos implementan:

```
class Book {
    final int id;
    final String title;
    final String author;
    // ... otros campos

    Book({
        required this.id,
        required this.title,
        required this.author,
    });

    factory Book.fromJson(Map<String, dynamic> json) {
        return Book(
            id: json['id'],
            title: json['title'],
            author: json['author'],
        );
    }

    Map<String, dynamic> toJson() {
        return {
            'id': id,
            'title': title,
            'author': author,
        };
    }
}
```

9.7. Cliente API (Dio)

9.7.1. Configuración

```
class ApiClient {
    static const String baseUrl = 'https://tu-servidor.com/api';
    late final Dio _dio;

    ApiClient() {
```

```
_dio = Dio(BaseOptions(  
  baseUrl: baseUrl,  
  connectTimeout: const Duration(seconds: 30),  
  receiveTimeout: const Duration(seconds: 30),  
  headers: {  
    'Content-Type': 'application/json',  
    'Accept': 'application/json',  
  },  
));  
  
_dio.interceptors.add(AuthInterceptor());  
_dio.interceptors.add(LogInterceptor());  
}  
  
Future<Response> get(String path) async {  
  return await _dio.get(path);  
}  
  
Future<Response> post(String path, {dynamic data}) async {  
  return await _dio.post(path, data: data);  
}  
  
// ... otros métodos  
}
```

9.7.2. Manejo de Errores

```
class ApiException implements Exception {  
  final int? statusCode;  
  final String message;  
  final dynamic data;  
  
  ApiException({  
    this.statusCode,  
    required this.message,  
    this.data,  
  });  
  
  @override  
  String toString() {  
    return 'ApiException($statusCode): $message';  
  }  
}
```

```
}  
}  
  
Future<T> handleApiCall<T>(Future<T> Function() call) async {  
  try {  
    return await call();  
  } on DioException catch (e) {  
    if (e.response != null) {  
      throw ApiException(  
        statusCode: e.response!.statusCode,  
        message: e.response!.data['detail'] ?? 'Error desconocido',  
        data: e.response!.data,  
      );  
    } else {  
      throw ApiException(  
        message: 'Error de conexión: ${e.message}',  
      );  
    }  
  }  
}
```

9.8. Almacenamiento Local

9.8.1. Flutter Secure Storage

```
class StorageService {  
  final FlutterSecureStorage _storage = const FlutterSecureStorage();  
  
  // Guardar token  
  Future<void> saveToken(String token) async {  
    await _storage.write(key: 'access_token', value: token);  
  }  
  
  // Leer token  
  Future<String?> getToken() async {  
    return await _storage.read(key: 'access_token');  
  }  
}
```

```
// Eliminar todo
Future<void> clearAll() async {
  await _storage.deleteAll();
}
}
```

9.9. Tematización

9.9.1. Tema de la Aplicación

```
class AppTheme {
  static ThemeData lightTheme = ThemeData(
    primarySwatch: Colors.blue,
    scaffoldBackgroundColor: Colors.white,
    appBarTheme: const AppBarTheme(
      backgroundColor: Colors.blue,
      foregroundColor: Colors.white,
      elevation: 0,
    ),
    elevatedButtonTheme: ElevatedButtonThemeData(
      style: ElevatedButton.styleFrom(
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
        padding: const EdgeInsets.symmetric(
          horizontal: 32,
          vertical: 16,
        ),
      ),
    ),
  );
}
```

9.10. Navegación

9.10.1. Sistema de Rutas

```
class AppRoutes {
  static const String login = '/login';
  static const String register = '/register';
  static const String home = '/home';
  static const String profile = '/profile';
  static const String bookDetail = '/book-detail';
  static const String pdfReader = '/pdf-reader';

  static Map<String, WidgetBuilder> routes = {
    login: (context) => const LoginPage(),
    register: (context) => const RegisterPage(),
    home: (context) => const HomePage(),
    profile: (context) => const ProfilePage(),
    // ...
  };
}
```

9.11. Testing

9.11.1. Estructura de Tests

```
test/
  unit/
    models/
      user_test.dart
      book_test.dart
    services/
      api_client_test.dart
    utils/
      validators_test.dart
  widget/
    book_card_test.dart
    rating_stars_test.dart
  integration/
    auth_flow_test.dart
```


9.11.2. Ejemplo de Test

```
void main() {
  group('Book Model', () {
    test('fromJson creates valid Book', () {
      final json = {
        'id': 1,
        'title': 'Test Book',
        'author': 'Test Author',
      };

      final book = Book.fromJson(json);

      expect(book.id, 1);
      expect(book.title, 'Test Book');
      expect(book.author, 'Test Author');
    });

    test('toJson creates valid JSON', () {
      final book = Book(
        id: 1,
        title: 'Test Book',
        author: 'Test Author',
      );

      final json = book.toJson();

      expect(json['id'], 1);
      expect(json['title'], 'Test Book');
      expect(json['author'], 'Test Author');
    });
  });
}
```

9.12. Performance y Optimización

9.12.1. Buenas Prácticas Implementadas

1. **Lazy Loading:** Carga de datos bajo demanda

2. **Caché de Imágenes:** Uso de `CachedNetworkImage`
3. **Paginación:** Listas largas con paginación
4. **Optimización de Builds:** Uso de `const` constructors
5. **Debouncing:** En búsquedas y filtros

9.12.2. Ejemplo de Optimización

```
// Widget con const constructor
class BookCard extends StatelessWidget {
  final Book book;

  const BookCard({
    Key? key,
    required this.book,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Card(
      child: ListTile(
        title: Text(book.title),
        subtitle: Text(book.author),
      ),
    );
  }
}

// Uso de const
const BookCard(book: myBook) // Evita reconstrucciones innecesarias
```

9.13. Seguridad

9.13.1. Medidas de Seguridad Implementadas

! Prácticas de Seguridad

1. **Tokens JWT:** Autenticación segura basada en tokens
2. **HTTPS:** Todas las comunicaciones cifradas
3. **Secure Storage:** Credenciales almacenadas de forma segura
4. **Validación de Entrada:** Sanitización de datos del usuario
5. **Timeout de Sesión:** Expiración automática de tokens
6. **No hay hardcoded secrets:** Uso de variables de entorno

9.13.2. Validación de Inputs

```
class Validators {
    static String? validateEmail(String? value) {
        if (value == null || value.isEmpty) {
            return 'El email es requerido';
        }
        final emailRegex = RegExp(r'^[\\w-\\.]+@([\\w-]+\\.)+[\\w-]{2,4}$');
        if (!emailRegex.hasMatch(value)) {
            return 'Email inválido';
        }
        return null;
    }

    static String? validatePassword(String? value) {
        if (value == null || value.isEmpty) {
            return 'La contraseña es requerida';
        }
        if (value.length < 8) {
            return 'Mínimo 8 caracteres';
        }
        return null;
    }
}
```

9.14. Configuración Multi-entorno

9.14.1. Entornos

```
enum Environment { development, staging, production }

class Config {
    static Environment environment = Environment.development;

    static String get apiBaseUrl {
        switch (environment) {
            case Environment.development:
                return 'http://localhost:8000/api';
            case Environment.staging:
                return 'https://staging.libris.app/api';
            case Environment.production:
                return 'https://api.libris.app';
        }
    }

    static Duration get apiTimeout {
        return environment == Environment.development
            ? const Duration(seconds: 60)
            : const Duration(seconds: 30);
    }
}
```

9.15. Build y CI/CD

9.15.1. Scripts de Build

```
# .github/workflows/build.yml
name: Build and Test

on: [push, pull_request]

jobs:
```

```
build:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v2
    - uses: subosito/flutter-action@v2
      with:
        flutter-version: '3.10.7'
    - run: flutter pub get
    - run: flutter analyze
    - run: flutter test
    - run: flutter build apk --release
```

9.16. Conclusión

La arquitectura de Libris está diseñada para ser:

- **Escalable:** Fácil de extender con nuevas funcionalidades
- **Mantenible:** Código organizado y bien estructurado
- **Testeable:** Separación clara de responsabilidades
- **Segura:** Implementación de mejores prácticas de seguridad
- **Performante:** Optimizaciones para una experiencia fluida

Para más detalles sobre componentes específicos, consulta:

- [API Reference](#)
- [Data Models](#)
- [Development Guide](#)

10. API Reference

Referencia completa de endpoints REST

11. API Reference

11.1. Base URL

Base URL: `http://tu-servidor.com/api`

⚠ Entornos

- **Desarrollo:** `http://localhost:8000/api`
- **Producción:** `https://api.libris.app`

Configura la URL correcta según tu entorno en `lib/api/api_client.dart`.

11.2. Autenticación

11.2.1. Sistema JWT

Libris usa autenticación basada en **JWT (JSON Web Tokens)**:

- **Access Token:** Vida corta (1 hora), usado en cada request
- **Refresh Token:** Vida larga (3 días), usado para renovar el access token

11.2.2. Headers Requeridos

Para endpoints autenticados, incluye:

```
Authorization: Bearer {access_token}
Content-Type: application/json
Accept: application/json
```

11.3. Endpoints

11.3.1. 1. Autenticación

11.3.1.1. 1.1 Login

Autentica un usuario y devuelve tokens JWT.

POST /api/login/

Request Body:

```
{
  "username": "juan_lector",
  "password": "mi_contraseña"
}
```

Response (200 OK):

```
{
  "access": "eyJ0eXAiOiJKV1QiLCJhbGc...",
  "refresh": "eyJ0eXAiOiJKV1QiLCJhbGc...",
  "user": {
    "id": 1,
    "username": "juan_lector",
    "email": "juan@email.com",
    "first_name": "Juan",
    "last_name": "Pérez"
  }
}
```

Errores:

Código	Descripción
400	Credenciales inválidas o campos faltantes
401	Usuario o contraseña incorrectos
500	Error interno del servidor

Ejemplo Flutter:


```
Future<Map<String, dynamic>> login(String username, String password) async {
  try {
    final response = await dio.post('/login/', data: {
      'username': username,
      'password': password,
    });

    // Guardar tokens
    await storage.write(key: 'access_token', value: response.data['access']);
    await storage.write(key: 'refresh_token', value: response.data['refresh']);

    return response.data['user'];
  } on DioException catch (e) {
    if (e.response?.statusCode == 401) {
      throw 'Credenciales incorrectas';
    }
    throw 'Error de conexión';
  }
}
```

11.3.1.2. 1.2 Registro

Crea una nueva cuenta de usuario.

POST /api/register/

Request Body:

```
{
  "username": "nuevo_usuario",
  "email": "usuario@email.com",
  "password": "contraseña_segura",
  "password2": "contraseña_segura",
  "first_name": "Nombre",
  "last_name": "Apellido"
}
```

Response (201 Created):

```
{
  "id": 2,
  "username": "nuevo_usuario",
  "email": "usuario@email.com",
  "first_name": "Nombre",
  "last_name": "Apellido",
  "message": "Usuario creado exitosamente"
}
```

Errores:

Código	Descripción
400	Validación fallida (contraseñas no coinciden, etc.)
409	Usuario o email ya existe

Validaciones:

- Username: único, alfanumérico, 3-150 caracteres
- Email: válido y único
- Password: mínimo 8 caracteres
- Password2: debe coincidir con password

11.3.1.3. 1.3 Refresh Token

Renueva el access token usando el refresh token.

POST /api/refresh/

Request Body:

```
{
  "refresh": "eyJ0eXAiOiJKV1QiLCJhbGc..."
}
```

Response (200 OK):

```
{
  "access": "eyJ0eXAiOiJKV1QiLCJhbGc..."
}
```

Errores:

Código	Descripción
401	Refresh token inválido o expirado

Uso Automático:

```
class AuthInterceptor extends Interceptor {
  @override
  Future<void> onError(DioException err, ErrorInterceptorHandler handler) async {
    if (err.response?.statusCode == 401) {
      // Token expirado, intentar refresh
      final refreshToken = await storage.read(key: 'refresh_token');

      try {
        final response = await dio.post('/refresh/', data: {
          'refresh': refreshToken,
        });

        await storage.write(key: 'access_token', value: response.data['access']);

        // Reintentar request original
        final opts = err.requestOptions;
        opts.headers['Authorization'] = 'Bearer ${response.data['access']}';
        final cloneReq = await dio.request(opts.path, options: Options(
          method: opts.method,
          headers: opts.headers,
        ));

        handler.resolve(cloneReq);
      } catch (e) {
        // Refresh falló, logout
        handler.next(err);
      }
    } else {
      handler.next(err);
    }
  }
}
```

11.3.2. 2. Catálogo

11.3.2.1. 2.1 Listar Libros

Obtiene el catálogo de libros.

```
GET /api/libros/
```

Headers:

Authorization: Bearer {access_token}

Query Parameters:

Parámetro	Tipo	Descripción
search	string	Buscar por título o autor
page	integer	Número de página (paginación)
page_size	integer	Items por página (default: 20)

Response (200 OK):

```
{
  "count": 150,
  "next": "http://api.libris.app/libros/?page=2",
  "previous": null,
  "results": [
    {
      "id": 1,
      "titulo": "El Quijote",
      "tipo": "Libro",
      "autor": "Miguel de Cervantes",
      "descripcion": "La historia del ingenioso hidalgo...",
      "portada": "https://api.libris.app/media/portadas/quijote.jpg",
      "archivo": "https://api.libris.app/media/libros/quijote.pdf",
      "contenido_pdf_url": "https://api.libris.app/media/libros/quijote.pdf",
      "numero_paginas": 863,
      "calificacion_promedio": 4.5,
      "created_at": "2024-01-01T00:00:00Z"
    }
  ]
}
```

Ejemplo con búsqueda:

```
Future<List<Book>> searchBooks(String query) async {  
  final response = await dio.get('/libros/', queryParameters: {  
    'search': query,  
  });  
  
  return (response.data['results'] as List)  
    .map((json) => Book.fromJson(json))  
    .toList();  
}
```

11.3.2.2. 2.2 Listar Mangas

```
GET /api/mangas/
```

Igual que /libros/ pero filtra por tipo: "Manga".

11.3.2.3. 2.3 Listar Novelas

```
GET /api/novelas/
```

Igual que /libros/ pero filtra por tipo: "Novela".

11.3.2.4. 2.4 Listar Material Educativo

```
GET /api/material/
```

Igual que /libros/ pero filtra por tipo: "Material".

11.3.3. 3. Mis Lecturas (Registros)

11.3.3.1. 3.1 Listar Mis Lecturas

Obtiene los registros de lectura del usuario autenticado.

```
GET /api/registros/
```

Headers:

Authorization: Bearer {access_token}

Response (200 OK):

```
[
  {
    "id": 1,
    "usuario": 1,
    "tipo": "libro",
    "libro": 5,
    "manga": null,
    "novela": null,
    "material": null,
    "estado": "leyendo",
    "progreso": 45,
    "pagina_actual": 135,
    "fecha_creacion": "2024-01-15T10:30:00Z",
    "fecha_actualizacion": "2024-02-01T14:20:00Z",
    "detalles_item": {
      "id": 5,
      "titulo": "El Quijote",
      "autor": "Miguel de Cervantes",
      "portada": "https://..."
    }
  }
]
```

Estados posibles:

- "leyendo" - En progreso
- "completado" - Terminado
- "pendiente" - Para leer después

- "abandonado" - Dejado

11.3.3.2. 3.2 Crear Registro

Agrega un libro a “Mis Lecturas”.

POST /api/registros/

Request Body:

```
{
  "tipo": "libro",
  "libro": 5,
  "estado": "leyendo"
}
```

Campos por tipo:

Tipo	Campo requerido
"libro"	libro: <id>
"manga"	manga: <id>
"novela"	novela: <id>
"material"	material: <id>

Response (201 Created):

```
{
  "id": 10,
  "usuario": 1,
  "tipo": "libro",
  "libro": 5,
  "estado": "leyendo",
  "fecha_creacion": "2024-02-05T10:00:00Z"
}
```

Errores:

11. API Reference

Código	Descripción
400	Datos inválidos (tipo o id faltante)
409	Ya existe un registro para este item

11.3.3.3. 3.3 Actualizar Registro

Actualiza el estado o progreso de una lectura.

```
PATCH /api/registros/{id}/
```

Request Body (parcial):

```
{
  "estado": "completado",
  "pagina_actual": 863
}
```

Response (200 OK):

```
{
  "id": 1,
  "estado": "completado",
  "progreso": 100,
  "pagina_actual": 863,
  "fecha_actualizacion": "2024-02-05T15:30:00Z"
}
```

11.3.3.4. 3.4 Eliminar Registro

Elimina un libro de “Mis Lecturas”.

```
DELETE /api/registros/{id}/
```


Response (204 No Content)

Sin cuerpo de respuesta.

Errores:

Código	Descripción
404	Registro no encontrado
403	No tienes permiso para eliminar este registro

11.3.4. 4. Calificaciones

11.3.4.1. 4.1 Listar Mis Calificaciones

```
GET /api/calificaciones/
```

Response (200 OK):

```
[
  {
    "id": 1,
    "usuario": 1,
    "tipo": "libro",
    "libro": 5,
    "puntuacion": 5,
    "fecha_creacion": "2024-01-20T10:00:00Z",
    "detalles_item": {
      "id": 5,
      "titulo": "El Quijote",
      "portada": "https://..."
    }
  }
]
```

11.3.4.2. 4.2 Crear Calificación

Califica un libro (1-5 estrellas).

```
POST /api/calificaciones/
```

Request Body:

```
{
  "tipo": "libro",
  "libro": 5,
  "puntuacion": 5
}
```

Validaciones:

- puntuacion: Entre 1 y 5 (entero)
- Solo una calificación por usuario/item

Response (201 Created):

```
{
  "id": 10,
  "usuario": 1,
  "tipo": "libro",
  "libro": 5,
  "puntuacion": 5,
  "fecha_creacion": "2024-02-05T10:00:00Z"
}
```

11.3.4.3. 4.3 Actualizar Calificación

Cambia la puntuación de una calificación existente.

```
PATCH /api/calificaciones/{id}/
```

Request Body:

```
{  
  "puntuacion": 4  
}
```

Response (200 OK):

```
{  
  "id": 10,  
  "puntuacion": 4,  
  "fecha_actualizacion": "2024-02-05T11:00:00Z"  
}
```

11.3.4.4. 4.4 Eliminar Calificación

```
DELETE /api/calificaciones/{id}/
```

Response (204 No Content)

11.3.5. 5. Comentarios

11.3.5.1. 5.1 Listar Comentarios de un Item

```
GET /api/comentarios/?tipo=libro&libro=5
```

Query Parameters:

Parámetro	Descripción
tipo	libro, manga, novela, o material
libro	ID del libro (si tipo=libro)
manga	ID del manga (si tipo=manga)
novela	ID de la novela (si tipo=novela)
material	ID del material (si tipo=material)

Response (200 OK):

```
[
  {
    "id": 1,
    "usuario": {
      "id": 1,
      "username": "juan_lector",
      "first_name": "Juan",
      "last_name": "Pérez"
    },
    "tipo": "libro",
    "libro": 5,
    "contenido": "¡Excelente libro! La narrativa es increíble...",
    "fecha_creacion": "2024-01-25T14:30:00Z",
    "fecha_actualizacion": null
  }
]
```

11.3.5.2. 5.2 Crear Comentario

POST /api/comentarios/

Request Body:

```
{
  "tipo": "libro",
  "libro": 5,
  "contenido": "Mi opinión sobre este libro..."
}
```

Validaciones:

- contenido: 10-1000 caracteres
- No HTML ni scripts

Response (201 Created):

```
{
  "id": 15,
  "usuario": {
    "id": 1,
    "username": "juan_lector"
  },
  "tipo": "libro",
  "libro": 5,
  "contenido": "Mi opinión sobre este libro...",
  "fecha_creacion": "2024-02-05T12:00:00Z"
}
```

11.3.5.3. 5.3 Actualizar Comentario

```
PATCH /api/comentarios/{id}/
```

Request Body:

```
{
  "contenido": "Actualizando mi comentario..."
}
```

Response (200 OK):

```
{
  "id": 15,
  "contenido": "Actualizando mi comentario...",
  "fecha_actualizacion": "2024-02-05T13:00:00Z"
}
```

11.3.5.4. 5.4 Eliminar Comentario

```
DELETE /api/comentarios/{id}/
```

Response (204 No Content)

11.4. Códigos de Estado HTTP

11.4.1. Exitosos (2xx)

Código	Significado	Uso
200 OK	Éxito	GET, PATCH exitosos
201 Created	Creado	POST exitoso
204 No Content	Sin contenido	DELETE exitoso

11.4.2. Errores del Cliente (4xx)

Código	Significado	Causa Común
400 Bad Request	Solicitud inválida	Validación fallida, datos faltantes
401 Unauthorized	No autorizado	Token expirado o inválido
403 Forbidden	Prohibido	Sin permisos para esta acción
404 Not Found	No encontrado	Recurso no existe
409 Conflict	Conflicto	Duplicado (ej: usuario ya existe)
422 Unprocessable Entity	No procesable	Validación semántica fallida

11.4.3. Errores del Servidor (5xx)

Código	Significado	Acción
500 Internal Server Error	Error del servidor	Reportar a soporte
502 Bad Gateway	Gateway incorrecto	Reintentar
503 Service Unavailable	Servicio no disponible	Reintentar más tarde

11.5. Manejo de Errores

11.5.1. Formato de Error

Todos los errores siguen este formato:

```
{
  "detail": "Mensaje de error legible",
  "code": "error_code",
  "errors": {
    "field_name": ["Error específico del campo"]
  }
}
```

11.5.2. Ejemplo en Flutter

```
Future<void> handleApiError(DioException e) {
  if (e.response != null) {
    final statusCode = e.response!.statusCode;
    final data = e.response!.data;

    switch (statusCode) {
      case 400:
        throw 'Datos inválidos: ${data['detail']}';
      case 401:
        // Token expirado, intentar refresh
        return refreshTokenAndRetry();
      case 403:
        throw 'No tienes permiso para esta acción';
      case 404:
        throw 'Recurso no encontrado';
      case 409:
        throw 'Este recurso ya existe';
      case 500:
        throw 'Error del servidor. Intenta más tarde';
      default:
        throw 'Error desconocido: $statusCode';
    }
  } else {
    throw 'Error de conexión: ${e.message}';
  }
}
```

11.6. Rate Limiting

Límites de Tasa

El API puede implementar rate limiting:

- **Autenticación:** 5 intentos/minuto
- **General:** 100 requests/minuto
- **Búsquedas:** 30 requests/minuto

Si excedes el límite, recibirás:

```
{
  "detail": "Demasiadas solicitudes",
  "retry_after": 60
}
```

Status Code: 429 Too Many Requests

11.7. Paginación

Los endpoints de listado usan paginación cursor:

```
{
  "count": 150,
  "next": "https://api.libris.app/libros/?page=2",
  "previous": null,
  "results": [...]
}
```

Navegar:

```
String? nextPage = response.data['next'];
if (nextPage != null) {
  final nextResponse = await dio.get(nextPage);
}
```


11.8. Búsqueda y Filtrado

Parámetros comunes de búsqueda:

Parámetro	Tipo	Ejemplo
search	string	?search=cervantes
ordering	string	?ordering=-created_at
page	integer	?page=2
page_size	integer	?page_size=50

Ordenamiento:

- Ascendente: `ordering=titulo`
 - Descendente: `ordering=-created_at`
-

11.9. CORS

El backend debe configurar CORS para permitir requests desde la app móvil:

```
# settings.py
CORS_ALLOWED_ORIGINS = [
    "http://localhost:8080",
    "https://app.libris.app",
]
```

11.10. Versionado

i Versión Actual: v1

La API actualmente no usa versionado en la URL. Futuras versiones podrían usar:

`/api/v2/libros/`

Los cambios breaking se comunicarán con anticipación.

11.11. Recursos Adicionales

- [Data Models](#): Estructuras de datos detalladas
- [HTTP Status Codes](#): Guía completa de códigos
- [Troubleshooting](#): Solución de problemas comunes

11.12. Changelog

11.12.1. v1.0.0 (2024-02-05)

- Endpoints iniciales de autenticación
- CRUD de lecturas
- CRUD de calificaciones
- CRUD de comentarios
- Catálogo de libros/mangas/novelas/material

12. Modelos de Datos

Estructuras de datos y esquemas JSON

13. Modelos de Datos

13.1. Introducción

Esta sección documenta todos los modelos de datos utilizados en Libris, incluyendo sus propiedades, tipos, validaciones y relaciones.

13.2. Modelos Principales

13.2.1. User (Usuario)

Representa un usuario registrado en el sistema.

Propiedades:

Campo	Tipo	Requerido	Descripción
id	integer	Sí	ID único del usuario
username	string	Sí	Nombre de usuario (único)
email	string	Sí	Email (único)
first_name	string	No	Nombre
last_name	string	No	Apellido
created_at	datetime	Sí	Fecha de registro
is_active	boolean	Sí	Usuario activo

Esquema JSON:

```
{
  "id": 1,
  "username": "juan_lector",
  "email": "juan@email.com",
  "first_name": "Juan",
  "last_name": "Pérez",
```

```

    "created_at": "2024-01-15T10:30:00Z",
    "is_active": true
}

```

Clase Dart:

```

class User {
  final int id;
  final String username;
  final String email;
  final String? firstName;
  final String? lastName;
  final DateTime createdAt;
  final bool isActive;

  User({
    required this.id,
    required this.username,
    required this.email,
    this.firstName,
    this.lastName,
    required this.createdAt,
    required this.isActive,
  });

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['id'],
      username: json['username'],
      email: json['email'],
      firstName: json['first_name'],
      lastName: json['last_name'],
      createdAt: DateTime.parse(json['created_at']),
      isActive: json['is_active'] ?? true,
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'id': id,
      'username': username,
      'email': email,
      'first_name': firstName,

```

```

        'last_name': lastName,
        'created_at': createdAt.toIso8601String(),
        'is_active': isActive,
    };
}

String get fullName => '${firstName ?? ''} ${lastName ?? ''}'.trim();
}

```

13.2.2. Book (Libro/Manga/Novela/Material)

Representa un ítem del catálogo.

Propiedades:

Campo	Tipo	Requerido	Descripción
id	integer	Sí	ID único
titulo	string	Sí	Título del libro
tipo	string	Sí	“Libro”, “Manga”, “Novela”, “Material”
autor	string	Sí	Autor
descripcion	string	No	Descripción
portada	string (URL)	No	URL de la portada
archivo	string (URL)	No	URL del PDF
contenido_pdf_url	string (URL)	No	URL del contenido PDF
numero_paginas	integer	No	Número de páginas
calificacion_promedio	float	No	Calificación promedio (1-5)
created_at	datetime	Sí	Fecha de creación

Esquema JSON:

```

{
  "id": 1,
  "titulo": "El Quijote",
  "tipo": "Libro",
  "autor": "Miguel de Cervantes",
  "descripcion": "La historia del ingenioso hidalgo Don Quijote de la Mancha...",
  "portada": "https://api.libris.app/media/portadas/quijote.jpg",
  "archivo": "https://api.libris.app/media/libros/quijote.pdf",
  "contenido_pdf_url": "https://api.libris.app/media/libros/quijote.pdf",

```

```

"numero_paginas": 863,
"calificacion_promedio": 4.5,
"created_at": "2024-01-01T00:00:00Z"
}

```

Clase Dart:

```

class Book {
  final int id;
  final String titulo;
  final String tipo;
  final String autor;
  final String? descripcion;
  final String? portada;
  final String? archivo;
  final String? contenidoPdfUrl;
  final int? numeroPaginas;
  final double? calificacionPromedio;
  final DateTime createdAt;

  Book({
    required this.id,
    required this.titulo,
    required this.tipo,
    required this.autor,
    this.descripcion,
    this.portada,
    this.archivo,
    this.contenidoPdfUrl,
    this.numeroPaginas,
    this.calificacionPromedio,
    required this.createdAt,
  });

  factory Book.fromJson(Map<String, dynamic> json) {
    return Book(
      id: json['id'],
      titulo: json['titulo'],
      tipo: json['tipo'],
      autor: json['autor'],
      descripcion: json['descripcion'],
      portada: json['portada'],
      archivo: json['archivo'],

```

```

    contenidoPdfUrl: json['contenido_pdf_url'],
    numeroPaginas: json['numero_paginas'],
    calificacionPromedio: json['calificacion_promedio']?.toDouble(),
    createdAt: DateTime.parse(json['created_at']),
  );
}

bool get hasPdf => archivo != null || contenidoPdfUrl != null;

String get pdfUrl => contenidoPdfUrl ?? archivo ?? '';
}

```

13.2.3. ReadingRecord (Registro de Lectura)

Representa el progreso de lectura de un usuario.

Propiedades:

Campo	Tipo	Requerido	Descripción
id	integer	Sí	ID único
usuario	integer	Sí	ID del usuario
tipo	string	Sí	“libro”, “manga”, “novela”, “material”
libro/manga/novela/material	integer	Condicional	ID del ítem (según tipo)
estado	string	Sí	Estado de lectura
progreso	integer	No	Porcentaje (0-100)
pagina_actual	integer	No	Página actual
fecha_creacion	datetime	Sí	Cuándo se agregó
fecha_actualizacion	datetime	Sí	Última actualización
detalles_item	object	No	Datos del libro/manga/etc

Estados válidos:

- "leyendo" - En progreso
- "completado" - Terminado
- "pendiente" - Para leer después
- "abandonado" - Dejado

Esquema JSON:

```
{
  "id": 1,
  "usuario": 1,
  "tipo": "libro",
  "libro": 5,
  "manga": null,
  "novela": null,
  "material": null,
  "estado": "leyendo",
  "progreso": 45,
  "pagina_actual": 135,
  "fecha_creacion": "2024-01-15T10:30:00Z",
  "fecha_actualizacion": "2024-02-01T14:20:00Z",
  "detalles_item": {
    "id": 5,
    "titulo": "El Quijote",
    "autor": "Miguel de Cervantes",
    "portada": "https://..."
  }
}
```

Clase Dart:

```
enum ReadingStatus {
  leyendo,
  completado,
  pendiente,
  abandonado;

  String get displayName {
    switch (this) {
      case ReadingStatus.leyendo:
        return 'Leyendo';
      case ReadingStatus.completado:
        return 'Completado';
      case ReadingStatus.pendiente:
        return 'Pendiente';
      case ReadingStatus.abandonado:
        return 'Abandonado';
    }
  }
}
```

```

}

class ReadingRecord {
    final int id;
    final int usuario;
    final String tipo;
    final int? libroId;
    final int? mangaId;
    final int? novelaId;
    final int? materialId;
    final ReadingStatus estado;
    final int? progreso;
    final int? paginaActual;
    final DateTime fechaCreacion;
    final DateTime fechaActualizacion;
    final Book? detallesItem;

    ReadingRecord({
        required this.id,
        required this.usuario,
        required this.tipo,
        this.libroId,
        this.mangaId,
        this.novelaId,
        this.materialId,
        required this.estado,
        this.progreso,
        this.paginaActual,
        required this.fechaCreacion,
        required this.fechaActualizacion,
        this.detallesItem,
    });

    factory ReadingRecord.fromJson(Map<String, dynamic> json) {
        return ReadingRecord(
            id: json['id'],
            usuario: json['usuario'],
            tipo: json['tipo'],
            libroId: json['libro'],
            mangaId: json['manga'],
            novelaId: json['novela'],
            materialId: json['material'],
            estado: ReadingStatus.values.firstWhere(

```

```

        (e) => e.name == json['estado'],
        orElse: () => ReadingStatus.pendiente,
    ),
    progreso: json['progreso'],
    paginaActual: json['pagina_actual'],
    fechaCreacion: DateTime.parse(json['fecha_creacion']),
    fechaActualizacion: DateTime.parse(json['fecha_actualizacion']),
    detallesItem: json['detalles_item'] != null
        ? Book.fromJson(json['detalles_item'])
        : null,
    );
}

int get itemId {
  switch (tipo) {
    case 'libro':
      return libroId!;
    case 'manga':
      return mangaId!;
    case 'novela':
      return novelaId!;
    case 'material':
      return materialId!;
    default:
      throw Exception('Tipo desconocido: $tipo');
  }
}
}
}

```

13.2.4. Rating (Calificación)

Representa una calificación de usuario para un ítem.

Propiedades:

Campo	Tipo	Requerido	Descripción
id	integer	Sí	ID único
usuario	integer	Sí	ID del usuario
tipo	string	Sí	“libro”, “manga”, “novela”, “material”

Campo	Tipo	Requerido	Descripción
libro/manga/novela/material	integer	Condicional	ID del ítem
puntuacion	integer	Sí	Calificación (1-5)
fecha_creacion	datetime	Sí	Cuándo se calificó
detalles_item	object	No	Datos del ítem

Esquema JSON:

```
{
  "id": 1,
  "usuario": 1,
  "tipo": "libro",
  "libro": 5,
  "manga": null,
  "novela": null,
  "material": null,
  "puntuacion": 5,
  "fecha_creacion": "2024-01-20T10:00:00Z",
  "detalles_item": {
    "id": 5,
    "titulo": "El Quijote",
    "portada": "https://..."
  }
}
```

Clase Dart:

```
class Rating {
  final int id;
  final int usuario;
  final String tipo;
  final int? libroId;
  final int? mangaId;
  final int? novelaId;
  final int? materialId;
  final int puntuacion; // 1-5
  final DateTime fechaCreacion;
  final Book? detallesItem;

  Rating({
    required this.id,
    required this.usuario,
```

```

        required this.tipo,
        this.libroId,
        this.mangaId,
        this.novelaId,
        this.materialId,
        required this.puntuacion,
        required this.fechaCreacion,
        this.detallesItem,
    });

    factory Rating.fromJson(Map<String, dynamic> json) {
        return Rating(
            id: json['id'],
            usuario: json['usuario'],
            tipo: json['tipo'],
            libroId: json['libro'],
            mangaId: json['manga'],
            novelaId: json['novela'],
            materialId: json['material'],
            puntuacion: json['puntuacion'],
            fechaCreacion: DateTime.parse(json['fecha_creacion']),
            detallesItem: json['detalles_item'] != null
                ? Book.fromJson(json['detalles_item'])
                : null,
        );
    }
}

Map<String, dynamic> toJson() {
    return {
        'tipo': tipo,
        if (libroId != null) 'libro': libroId,
        if (mangaId != null) 'manga': mangaId,
        if (novelaId != null) 'novela': novelaId,
        if (materialId != null) 'material': materialId,
        'puntuacion': puntuacion,
    };
}
}

```

13.2.5. Comment (Comentario)

Representa un comentario de usuario sobre un ítem.

Propiedades:

Campo	Tipo	Requerido	Descripción
id	integer	Sí	ID único
usuario	object	Sí	Datos del usuario autor
tipo	string	Sí	“libro”, “manga”, “novela”, “material”
libro/manga/novela/material	integer	Condicional	ID del ítem
contenido	string	Sí	Texto del comentario
fecha_creacion	datetime	Sí	Cuándo se creó
fecha_actualizacion	datetime	No	Última edición

Esquema JSON:

```
{
  "id": 1,
  "usuario": {
    "id": 1,
    "username": "juan_lector",
    "first_name": "Juan",
    "last_name": "Pérez"
  },
  "tipo": "libro",
  "libro": 5,
  "manga": null,
  "novela": null,
  "material": null,
  "contenido": "¡Excelente libro! La narrativa es increíble...",
  "fecha_creacion": "2024-01-25T14:30:00Z",
  "fecha_actualizacion": null
}
```

Clase Dart:

```
class Comment {
  final int id;
  final User usuario;
  final String tipo;
}
```

```

final int? libroId;
final int? mangaId;
final int? novelaId;
final int? materialId;
final String contenido;
final DateTime fechaCreacion;
final DateTime? fechaActualizacion;

Comment({
  required this.id,
  required this.usuario,
  required this.tipo,
  this.libroId,
  this.mangaId,
  this.novelaId,
  this.materialId,
  required this.contenido,
  required this.fechaCreacion,
  this.fechaActualizacion,
});

factory Comment.fromJson(Map<String, dynamic> json) {
  return Comment(
    id: json['id'],
    usuario: User.fromJson(json['usuario']),
    tipo: json['tipo'],
    libroId: json['libro'],
    mangaId: json['manga'],
    novelaId: json['novela'],
    materialId: json['material'],
    contenido: json['contenido'],
    fechaCreacion: DateTime.parse(json['fecha_creacion']),
    fechaActualizacion: json['fecha_actualizacion'] != null
      ? DateTime.parse(json['fecha_actualizacion'])
      : null,
  );
}

Map<String, dynamic> toJson() {
  return {
    'tipo': tipo,
    if (libroId != null) 'libro': libroId,
    if (mangaId != null) 'manga': mangaId,
  };
}

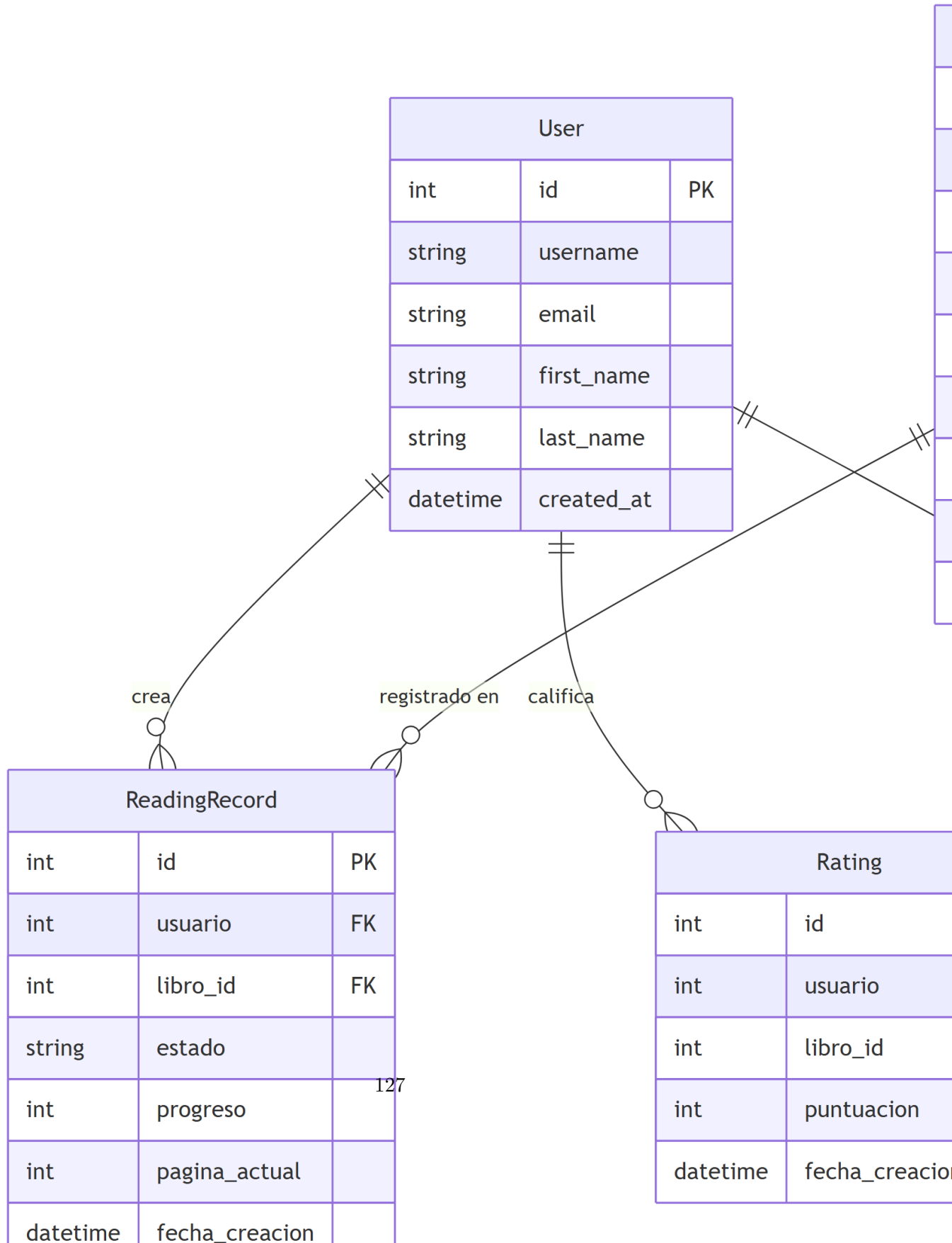
```

```
        if (novelaId != null) 'novela': novelaId,  
        if (materialId != null) 'material': materialId,  
        'contenido': contenido,  
    };  
}  
  
bool get wasEdited => fechaActualizacion != null;  
}
```

13. Modelos de Datos

13.3. Diagramas de Relaciones

13.3.1. Diagrama ER



13.4. Validaciones

13.4.1. User

```
class UserValidators {
    static String? validateUsername(String? value) {
        if (value == null || value.isEmpty) {
            return 'El usuario es requerido';
        }
        if (value.length < 3) {
            return 'Mínimo 3 caracteres';
        }
        if (value.length > 150) {
            return 'Máximo 150 caracteres';
        }
        if (!RegExp(r'^[a-zA-Z0-9_]+$').hasMatch(value)) {
            return 'Solo letras, números y guión bajo';
        }
        return null;
    }

    static String? validateEmail(String? value) {
        if (value == null || value.isEmpty) {
            return 'El email es requerido';
        }
        final emailRegex = RegExp(r'^[\w-\.] + @ ( [\w-] + \. ) + [ \w- ] { 2, 4 } $');
        if (!emailRegex.hasMatch(value)) {
            return 'Email inválido';
        }
        return null;
    }

    static String? validatePassword(String? value) {
        if (value == null || value.isEmpty) {
            return 'La contraseña es requerida';
        }
        if (value.length < 8) {
            return 'Mínimo 8 caracteres';
        }
    }
}
```

```
    return null;
  }
}
```

13.4.2. Rating

```
class RatingValidators {
    static String? validateScore(int? value) {
        if (value == null) {
            return 'La puntuación es requerida';
        }
        if (value < 1 || value > 5) {
            return 'La puntuación debe estar entre 1 y 5';
        }
        return null;
    }
}
```

13.4.3. Comment

```
class CommentValidators {
    static String? validateContent(String? value) {
        if (value == null || value.isEmpty) {
            return 'El comentario es requerido';
        }
        if (value.length < 10) {
            return 'Mínimo 10 caracteres';
        }
        if (value.length > 1000) {
            return 'Máximo 1000 caracteres';
        }
        return null;
    }
}
```

13.5. Serialización y Deserialización

13.5.1. Manejo de Nullables

```
// Manejo seguro de valores opcionales
class Book {
  factory Book.fromJson(Map<String, dynamic> json) {
    return Book(
      id: json['id'] as int,
      titulo: json['titulo'] as String,
      // Valores opcionales con valores por defecto
      descripcion: json['descripcion'] as String?,
      // Conversión segura de números
      calificacionPromedio: (json['calificacion_promedio'] as num?)?.toDouble(),
      // Parsing de fechas con manejo de errores
      createdAt: json['created_at'] != null
        ? DateTime.tryParse(json['created_at'] as String) ?? DateTime.now()
        : DateTime.now(),
    );
  }
}
```

13.5.2. JSON Encoding

```
import 'dart:convert';

// Serializar a JSON string
String bookJson = jsonEncode(book.toJson());

// Deserializar desde JSON string
Map<String, dynamic> jsonMap = jsonDecode(bookJson);
Book book = Book.fromJson(jsonMap);
```

13.6. Constantes y Enums

```
// Estados de lectura
enum ReadingStatus {
    leyendo,
    completado,
    pendiente,
    abandonado,
}

// Tipos de contenido
enum ContentType {
    libro,
    manga,
    novela,
    material,
}

// Constantes
class BookConstants {
    static const int minRating = 1;
    static const int maxRating = 5;
    static const int minCommentLength = 10;
    static const int maxCommentLength = 1000;
    static const int minUsernameLength = 3;
    static const int maxUsernameLength = 150;
    static const int minPasswordLength = 8;
}
```

13.7. Recursos Adicionales

- [API Reference](#): Endpoints completos
- [Technical Docs](#): Arquitectura de la app
- [Development Guide](#): Guía de desarrollo

14. Arquitectura

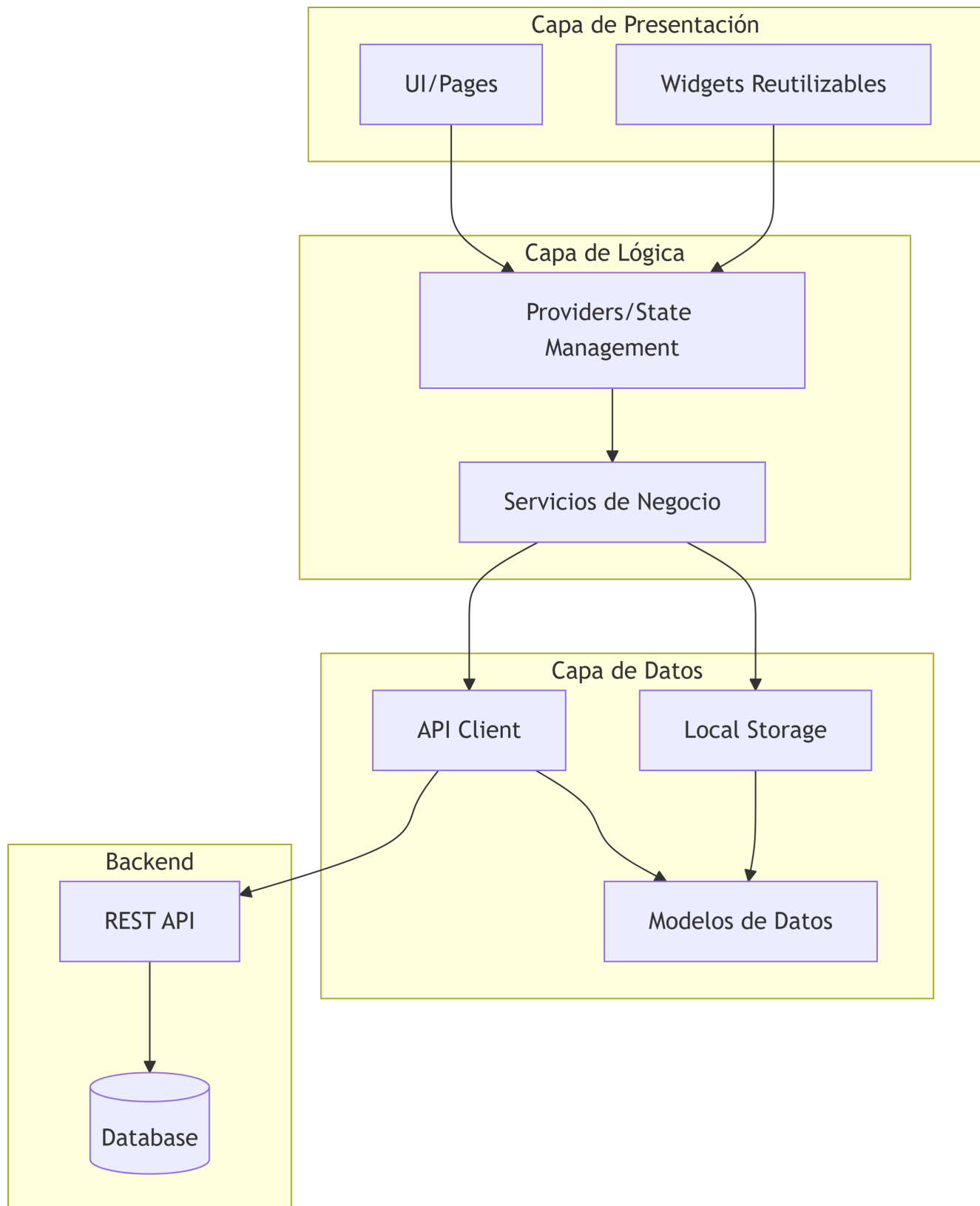
Diseño y patrones de arquitectura

15. Arquitectura del Sistema

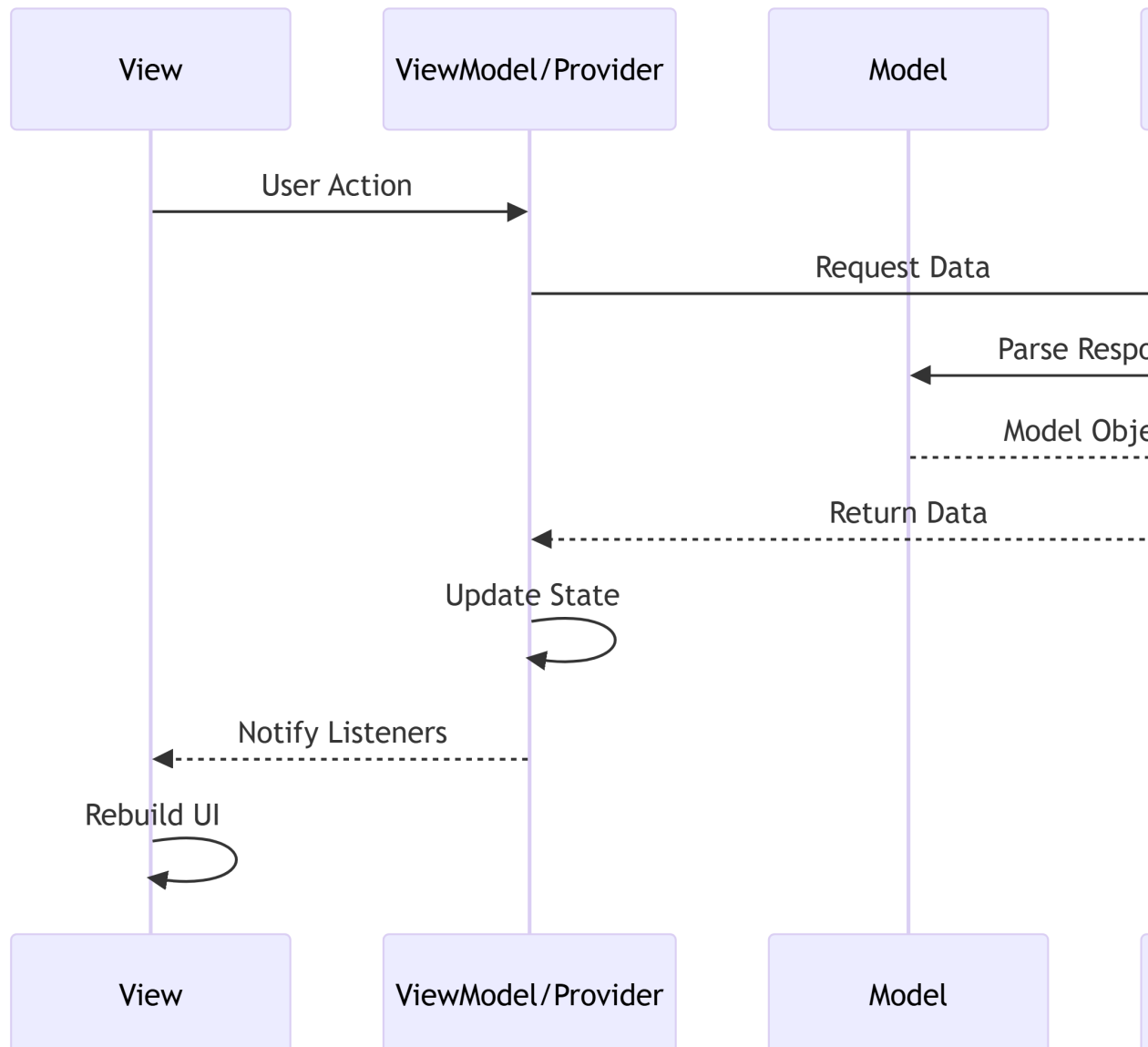
15.1. Visión General

Libris sigue una arquitectura de **3 capas** con separación clara de responsabilidades:

15. Arquitectura del Sistema



15.2. Patrón MVVM



15.3. Gestión de Estado con Provider

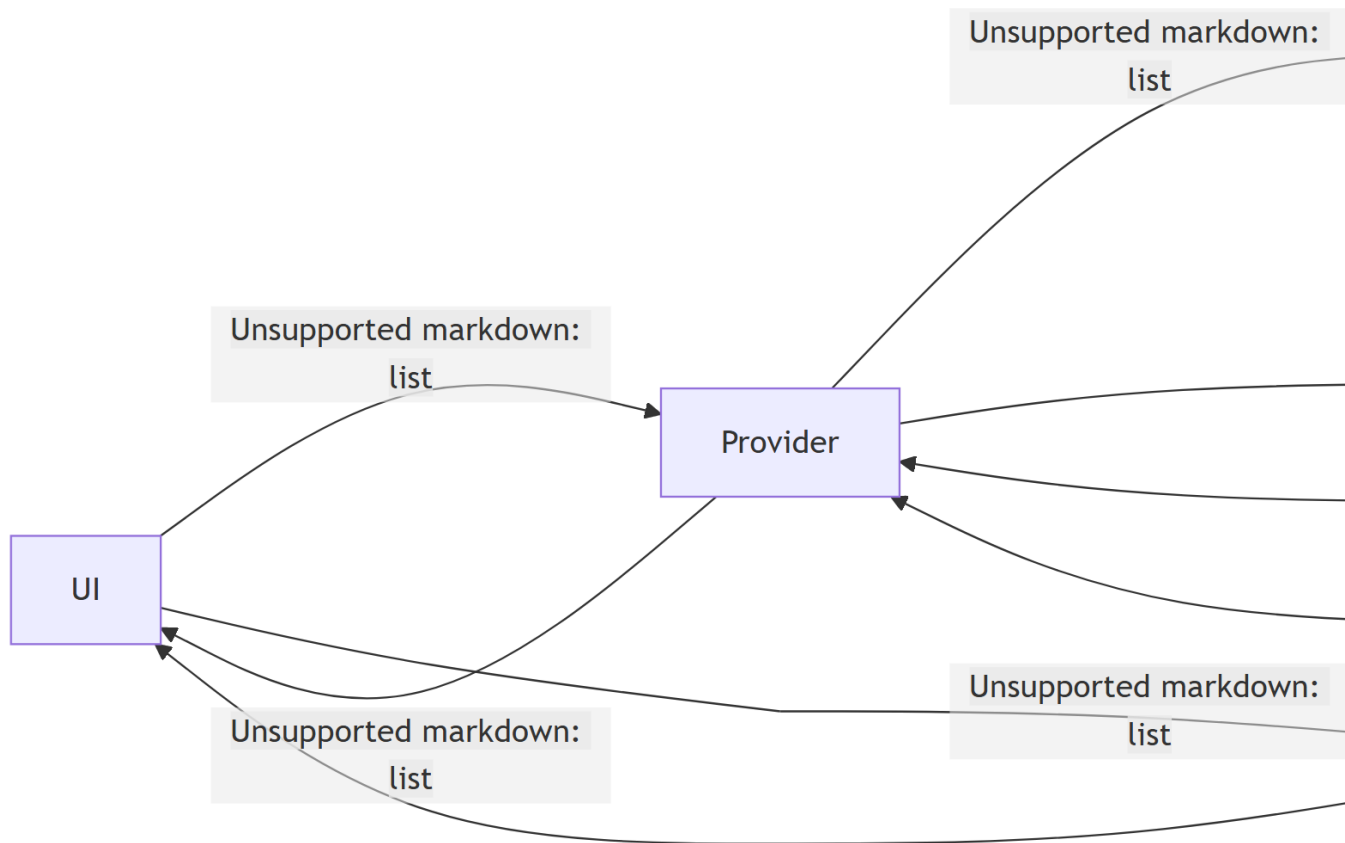
```
// Estructura de providers
void main() {
  runApp(
```

15. Arquitectura del Sistema

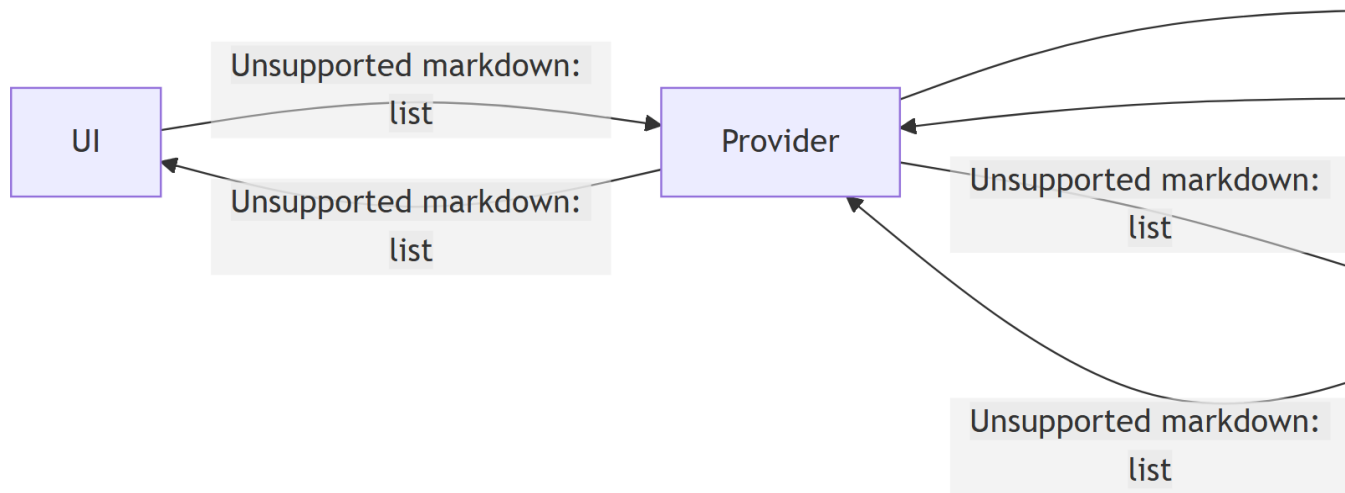
```
MultiProvider(  
  providers: [  
    ChangeNotifierProvider(create: (_) => AuthProvider()),  
    ChangeNotifierProvider(create: (_) => BookProvider()),  
    ChangeNotifierProvider(create: (_) => UserProvider()),  
  ],  
  child: MyApp(),  
)  
);  
}
```

15.4. Flujo de Datos

15.4.1. Lectura de Datos (Query)



15.4.2. Escritura de Datos (Mutation)



15.5. Inyección de Dependencias

```

class ServiceLocator {
    static final ServiceLocator _instance = ServiceLocator._internal();
    factory ServiceLocator() => _instance;
    ServiceLocator._internal();

    // Singleton instances
    final ApiClient apiClient = ApiClient();
    final StorageService storage = StorageService();

    // Lazy singletons
    late final AuthService authService = AuthService(apiClient, storage);
    late final BookService bookService = BookService(apiClient);
}

// Uso
final locator = ServiceLocator();
final books = await locator.bookService.getBooks();
  
```

15.6. Navegación

```
class AppRouter {
  static Route<dynamic> generateRoute(RouteSettings settings) {
    switch (settings.name) {
      case '/':
        return MaterialPageRoute(builder: (_) => LoginPage());
      case '/home':
        return MaterialPageRoute(builder: (_) => HomePage());
      case '/book-detail':
        final book = settings.arguments as Book;
        return MaterialPageRoute(
          builder: (_) => BookDetailPage(book: book),
        );
      default:
        return MaterialPageRoute(
          builder: (_) => Scaffold(
            body: Center(child: Text('No route defined for ${settings.name}')),
          ),
        );
    }
  }
}
```

15.7. Seguridad

15.7.1. Almacenamiento Seguro de Tokens

```
class SecureTokenStorage {
  final FlutterSecureStorage _storage = const FlutterSecureStorage();

  Future<void> saveTokens({
    required String accessToken,
    required String refreshToken,
  }) async {
    await Future.wait([
      _storage.write(key: 'access_token', value: accessToken),
      _storage.write(key: 'refresh_token', value: refreshToken),
    ]);
  }
}
```

```

Future<String?> getAccessToken() async {
  return await _storage.read(key: 'access_token');
}

Future<void> clearTokens() async {
  await _storage.deleteAll();
}
}

```

15.8. Manejo de Errores

```

class ErrorHandler {
  static void handleError(dynamic error, StackTrace stackTrace) {
    if (error is DioException) {
      _handleDioError(error);
    } else if (error is ApiException) {
      _handleApiError(error);
    } else {
      _handleGenericError(error, stackTrace);
    }
  }

  static void _handleDioError(DioException error) {
    switch (error.type) {
      case DioExceptionType.connectionTimeout:
      case DioExceptionType.receiveTimeout:
        showSnackBar('Tiempo de espera agotado');
        break;
      case DioExceptionType.badResponse:
        final statusCode = error.response?.statusCode;
        _handleHttpError(statusCode, error.response?.data);
        break;
      default:
        showSnackBar('Error de conexión');
    }
  }
}

```

15.9. Caché y Optimización

```
class CacheManager {
  final Map<String, CacheEntry> _cache = {};
  final Duration defaultTTL = Duration(minutes: 5);

  T? get<T>(String key) {
    final entry = _cache[key];
    if (entry == null || entry.isExpired) {
      _cache.remove(key);
      return null;
    }
    return entry.data as T;
  }

  void set<T>(String key, T data, {Duration? ttl}) {
    _cache[key] = CacheEntry(
      data: data,
      expiresAt: DateTime.now().add(ttl ?? defaultTTL),
    );
  }

  void clear() => _cache.clear();
}

class CacheEntry {
  final dynamic data;
  final DateTime expiresAt;

  CacheEntry({required this.data, required this.expiresAt});

  bool get isExpired => DateTime.now().isAfter(expiresAt);
}
```

15.10. Testing

15.10.1. Estructura de Tests

```
test/
  unit/
    models/
```



```
services/  
validators/  
widget/  
pages/  
integration/  
flows/
```

15.10.2. Mocking

```
class MockApiClient extends Mock implements ApiClient {}  
  
void main() {  
  group('BookProvider', () {  
    late MockApiClient mockApi;  
    late BookProvider provider;  
  
    setUp(() {  
      mockApi = MockApiClient();  
      provider = BookProvider(mockApi);  
    });  
  
    test('fetchBooks actualiza lista correctamente', () async {  
      when(mockApi.get('/libros/')).thenAnswer(  
        (_) async => Response(  
          data: {'results': [/* mock data */]},  
          statusCode: 200,  
        ),  
      );  
  
      await provider.fetchBooks();  
  
      expect(provider.books.length, greaterThan(0));  
      expect(provider.isLoading, false);  
    });  
  });  
}
```

Ver más en [Development Guide](#).

Part III.

Guias de Desarrollo

16. Guía de Desarrollo

Configuración del entorno y workflow de desarrollo

17. Guía de Desarrollo

Esta guía te ayudará a configurar tu entorno de desarrollo y contribuir al proyecto Libris.

17.1. Prerrequisitos

Asegúrate de tener instalado:

- Flutter 3.10.7 o superior
- Dart SDK (incluido con Flutter)
- Android Studio o VS Code
- Git

Ver [Guía de Instalación](#) para detalles completos.

17.2. Clonar el Repositorio

```
git clone https://github.com/tu-usuario/libris.git
cd libris/libris
flutter pub get
```

17.3. Estructura de Branches

- main - Producción (protegida)
- develop - Desarrollo activo
- feature/* - Nuevas funcionalidades
- bugfix/* - Correcciones de bugs
- hotfix/* - Correcciones urgentes

17.4. Workflow de Desarrollo

1. Crear una rama desde develop:

```
git checkout develop
git pull origin develop
git checkout -b feature/nombre-feature
```

2. Desarrollar la funcionalidad

3. Commitear cambios:

```
git add .
git commit -m "feat: descripción del cambio"
```

4. Push y crear PR:

```
git push origin feature/nombre-feature
```

17.5. Convenciones de Código

17.5.1. Formato

```
# Formatear código
flutter format lib/

# Analizar código
flutter analyze
```

17.5.2. Nombrado

```
// Clases: PascalCase
class BookProvider extends ChangeNotifier {}

// Métodos y variables: camelCase
void fetchBooks() {}
String userName = '';

// Constantes: lowerCamelCase
```

```
const String apiBaseUrl = 'https://api.libris.app';

// Archivos: snake_case
book_detail_page.dart
api_client.dart
```

17.5.3. Imports

```
// 1. Dart SDK
import 'dart:async';

// 2. Flutter
import 'package:flutter/material.dart';

// 3. Paquetes externos
import 'package:provider/provider.dart';
import 'package:dio/dio.dart';

// 4. Archivos locales
import '../models/book.dart';
import '../services/api_client.dart';
```

17.6. Testing

17.6.1. Ejecutar Tests

```
# Todos los tests
flutter test

# Tests específicos
flutter test test/unit/models/book_test.dart

# Con cobertura
flutter test --coverage
```

17.6.2. Escribir Tests

```
void main() {  
  group('Book Model', () {  
    test('fromJson crea Book válido', () {  
      final json = {'id': 1, 'titulo': 'Test'};  
      final book = Book.fromJson(json);  
  
      expect(book.id, 1);  
      expect(book.titulo, 'Test');  
    });  
  });  
}
```

17.7. Debug

17.7.1. Ejecutar en modo debug

```
flutter run
```

17.7.2. Hot Reload

Durante la ejecución:

- r - Hot reload
- R - Hot restart
- q - Quit

17.7.3. DevTools

```
flutter pub global activate devtools  
flutter pub global run devtools
```

17.8. Build

17.8.1. Debug Build

```
flutter build apk --debug
```

17.8.2. Release Build

```
flutter build apk --release
```

17.9. Contribuir

Ver [Contributing Guide](#) para detalles completos.

18. Solución de Problemas

Diagnóstico y resolución de problemas comunes

19. Troubleshooting

19.1. Problemas de Instalación

19.1.1. Flutter Doctor muestra errores

```
flutter doctor -v
```

Soluciones comunes:

- Android licenses: `flutter doctor --android-licenses`
- CocoaPods (iOS): `sudo gem install cocoapods`
- PATH no configurado: Agregar Flutter al PATH del sistema

19.2. Problemas de Compilación

19.2.1. Gradle build failed

```
cd android
./gradlew clean
cd ..
flutter clean
flutter pub get
```

19.2.2. Pod install failed (iOS)

```
cd ios
pod deintegrate
pod repo update
pod install
cd ..
```

19.3. Problemas de Red

19.3.1. Connection timeout

Verificar:

1. Conexión a Internet activa
2. URL del API correcta en `api_client.dart`
3. Firewall no bloquea la conexión
4. Certificado SSL válido

19.3.2. 401 Unauthorized constante

Causa: Token expirado o inválido

Solución:

```
// Limpiar tokens y volver a hacer login
await storage.deleteAll();
// Navegar a pantalla de login
```

19.4. Problemas de Estado

19.4.1. UI no se actualiza

Causa: No se está llamando `notifyListeners()`

Solución:

```
class MyProvider extends ChangeNotifier {
  void updateData() {
    // ... actualizar datos
    notifyListeners(); // ← No olvidar
  }
}
```

19.5. Logs y Debugging

```
import 'package:flutter/foundation.dart';

debugPrint('Debug message');
print('Standard output');
```

19.6. Obtener Ayuda

- GitHub Issues: github.com/tu-usuario/libris/issues
- Email: soporte@libris.app
- Documentación: docs.libris.app

20. Guía de Contribución

Cómo contribuir al proyecto Libris

21. Contributing Guide

¡Gracias por tu interés en contribuir a Libris! Esta guía te ayudará a empezar.

21.1. Código de Conducta

- Sé respetuoso con todos
- Usa lenguaje inclusivo
- Acepta críticas constructivas
- Enfócate en lo mejor para la comunidad

21.2. Cómo Contribuir

21.2.1. Reportar Bugs

1. Verifica que el bug no esté ya reportado
2. Crea un issue con el template de bug
3. Incluye:
 - Descripción clara
 - Pasos para reproducir
 - Comportamiento esperado vs actual
 - Screenshots si aplica
 - Versión de la app y dispositivo

21.2.2. Solicitar Features

1. Verifica que no exista ya la solicitud
2. Crea un issue con el template de feature
3. Describe claramente el problema que resuelve
4. Propón una solución si es posible

21.2.3. Pull Requests

1. **Fork el repositorio**
2. **Crea una rama** desde develop:

```
git checkout -b feature/mi-feature
```

3. **Desarrolla** siguiendo las **convenciones de código**
4. **Escribe tests** para tu código
5. **Commitea** con mensajes descriptivos:

```
git commit -m "feat: agregar función de búsqueda"
```

6. **Push** a tu fork:

```
git push origin feature/mi-feature
```

7. **Crea un PR** hacia develop

21.3. Convenciones de Commit

Usamos [Conventional Commits](#):

- **feat**: Nueva funcionalidad
- **fix**: Corrección de bug
- **docs**: Cambios en documentación
- **style**: Formato, sin cambios de código
- **refactor**: Refactorización de código
- **test**: Agregar o actualizar tests
- **chore**: Mantenimiento general

Ejemplos:

```
feat: agregar página de búsqueda
fix: corregir error de login
docs: actualizar README
refactor: mejorar estructura de providers
test: agregar tests para BookProvider
```

21.4. Checklist para PR

Antes de enviar tu PR, verifica:

- ☐ El código sigue las convenciones del proyecto
- ☐ Has agregado/actualizado tests
- ☐ Todos los tests pasan (`flutter test`)
- ☐ El código está formateado (`flutter format`)
- ☐ No hay warnings (`flutter analyze`)
- ☐ Has actualizado la documentación si es necesario
- ☐ El commit message es descriptivo

21.5. Revisión de Código

- Los PRs requieren al menos 1 aprobación
- Responde a los comentarios constructivamente
- Haz los cambios solicitados
- Una vez aprobado, será merged por un maintainer

21.6. Contacto

- Email: dev@libris.app
- GitHub Discussions: github.com/tu-usuario/libris/discussions

¡Gracias por contribuir!